

学 号：	519club
------	---------

武汉理工大学

软件工程实践（二）

学 院	计算机与人工智能学院
专 业	软件工程
班 级	软件 2301 班
姓 名	肖梦琪、刘晶、彭慧星、庞绮君
编 号	519club
指导教师	唐祖锴

2026 年 6 月 20 日

目录

1. 任务概述.....	1
2. 小组任务计划.....	2
2.1. 任务分析.....	2
2.2. 项目架构与功能设计	3
2.3. 团队组成与分工	7
2.4. 软件工具与项目约束	9
3. 小组软件过程.....	10
3.1. 代码仓库.....	10
3.2. 软件版本计划.....	10
3.3. 开发分支模型与提交流程.....	11
3.4. 代码规范与检查流程.....	12
3.5. 项目测试计划与实施流程.....	12
3.6. 项目集成与发布规范	14
3.7. AI 辅助完成的开发工作	14
4. 小组成员任务报告	16
4.1. 成员一任务分工及实施过程描述.....	16
4.2. 成员二任务分工及实施过程描述.....	32
4.3. 成员三任务分工及实施过程描述.....	50
4.4. 成员四任务分工及实施过程描述.....	53
5. 实施过程问题记录与分析	56
6. 任务总结.....	61
7. 参考文献.....	62

1. 任务概述

本次实践项目《熄灯前归寝》是在经典文本冒险游戏 World of Zuul 基础上进行的大型功能扩展与主题重构。项目核心目标是将原作的简单迷宫探索改造为一个具有完整叙事逻辑、五层递进关卡、丰富物品交互和严格时间压力的闯关解密游戏。玩家扮演一名晚归的学生，必须在宿舍楼 23:00 熄灯之前，从校园正门出发，依次通过五个难度递增的关卡，最终成功返回寝室执行 sleep 命令通关。

项目开发严格遵循软件工程规范，重点实践了以下核心模块的开发与集成。核心玩法与逻辑扩展（E1—E17）：在已实现的实训基础功能（房间物品、多步回退、随机传送、玩家负重系统）之上，为五关玩法新增并实现了全套拓展命令（use、submit、combine、unlock、feed、sleep 等）、关卡管理器（LevelManager）、熄灯计时器（LevelTimer）、操作耗时系统、条件门禁、黑暗区域惩罚以及按关卡解锁地图等核心机制，构建了完整的游戏逻辑闭环。高级功能实现（F6—F8）：项目同步开发并集成了基于 Spring Boot REST API 的简化网络多人模式（F6）、基于 Vue 3 + TypeScript 的图形化前端 vue-portal（F7，含实时倒计时、背包展示、经典与沉浸双视图），以及基于 H2 嵌入式数据库的游戏进度存档与读档功能（F8）。

软件工程实践：整个开发周期采用四层架构（表现层 Vue、应用层 Spring Boot、领域层 Java 游戏内核、基础设施层 H2）与命令模式，游戏规则集中在领域层，Spring Boot 通过 Facade 调用内核、不在 Web 层重复逻辑。项目严格遵循 Git Flow 简化版分支管理策略，通过 GitHub Issues 进行任务跟踪，利用 Pull Request 进行代码评审，并依托 GitHub Actions 构建了包含静态检查（Checkstyle）、单元测试（JUnit 5）和自动化打包（Maven）的持续集成流水线，确保项目高质量、可追溯、可部署。

本报告将详细阐述小组在项目开发全过程中的任务计划、软件过程管理、各成员具体贡献、问题记录与解决方案，并对整体实践成果进行总结。

2. 小组任务计划

2.1. 任务分析

本次实训是在软件工程实践一完成的 World of Zuul 样例工程基础上,以我们四人小组为单位进行协同开发。课程要求我们在保留原有文本冒险框架的前提下,扩充功能、规范代码、引入 Maven 与 GitHub 协作流程,并尝试 DevOps 中的自动化检查、测试与打包。我们小组没有另起炉灶做全新游戏,而是把选题定为《熄灯前归寝》——玩家扮演深夜从校门赶回寝室的学生,在熄灯倒计时内完成换卡、取归寝单、解谜、提交物品等操作,最终在寝室执行 sleep 过关。这样既保留了 Zuul 的命令驱动结构,又具备完整交互界面和可演示的五关流程,符合课程对“完整交互界面和功能逻辑”的要求。

从工作重心来看,我们把任务分成三块。第一块是实训基础功能 F1—F5,包括房间物品与 look、单步/多步 back、随机传送房间、玩家背包与 take/drop/items/eat 等,这部分在进组前已基本完成,我们后续只做维护和联调修补。第二块是本作拓展 E1—E17,是我们本次开发的主战场:新增 use、submit、combine、unlock、feed、sleep 等命令,配套五关进度管理、熄灯计时、操作罚时、条件门、西楼困锁、按关解锁房间等规则。第三块是实训高级功能 F6—F8,即 Spring Boot 联机服务端、Vue 3 图形前端、H2 本地存档与用户认证,用来体现课程强调的 Spring MVC、RESTful API、Vue 和数据库持久化。

开发节奏上,我们把 6 月 1 日至 6 月 20 日拆成四个阶段:6 月 1—7 日搭关卡与命令骨架,打通第 1—2 关;6 月 8—14 日补齐 E 系列命令与五关配置,同步启动 Vue 界面;6 月 15—18 日集中做 F6/F7/F8 联调并冻结功能;6 月 19—20 日补测试、写报告、录演示视频、从 dev 合并到 master 发布。我们在 GitHub 上把任务拆成 27 个 Issue,每个 Issue 指定负责人、依赖关系和验收标准,PR 合并时写 Closes #xx 关闭对应任务,方便老师按仓库记录核对每人工作量。

表 2-1 开发阶段与里程碑

阶段	时间	主要目标	阶段完成标志
迭代一: 框架搭建	6/1—6/7	搭关卡与命令骨架, 打通第 1—2 关	第 1 关“换卡+sleep”可玩
迭代二: 功能补齐	6/8—6/14	补齐 E 系列命令与五关配置, 启动 Vue	五关单机流程贯通

迭代三：集成联调	6/15—6/18	F6/F7/F8 联调，6/18 功能冻结	联机可演示，H2 可存档
迭代四：测试发布	6/19—6/20	测试、报告、视频、发布	master 答辩稳定版

重难点主要有四处。一是计时与关卡状态的一致性：LevelTimer 管“距熄灯（23:00）还有 XXX 秒”，各命令通过 ActionTimeCost 扣秒，联机模式下还要以服务端计时为准，前后端不能各算各的。二是命令之间的依赖链：比如第 2 关要先 submit 归寝单才能进寝室，第 3 关西楼要先 combine 再 use 砸锁，Issue 依赖关系必须按顺序开发，否则联调会反复返工。三是混合架构下的职责划分：我们把游戏规则只写在 Java 内核，Spring 只做 REST 桥接，Vue 只做展示和发命令，避免在 Web 层重复实现五关逻辑。四是多人协作时的合并冲突：Game.java、CommandWords.java、pom.xml 是冲突高发区，我们约定固定个人分支、小步提交、每日同步 dev 来降低风险。

2.2. 项目架构与功能设计

2.2.1 需求分析

我们在 6 月 3 日小组讨论后形成《需求说明文档》，把功能需求和非功能需求分开写清楚，作为后续架构设计和 Issue 拆分的依据。

功能性需求方面，我们把游戏定义为闯关解密类文字冒险：玩家从校门出发，在熄灯前集齐凭证回寝室 sleep 过关。核心玩法包括房间探索、物品拾取与负重管理、方向移动与 back 回退、拓展命令交互、五关递进解锁、NPC 对话、干扰物与公告提示等。五关各有差异化目标：第 1 关熟悉换卡流程；第 2 关引入归寝单与 submit；第 3 关增加主楼断电与西楼困锁解谜；第 4 关开放图书馆领单与喂猫支线；第 5 关增加智能锁密码与体育馆随机传送。除单机外，我们还要求 Vue 图形界面覆盖全部指令、H2 本地存档读档、Spring Boot 联机房间与状态同步。

非功能性需求方面，我们强调计时规则必须严格一致：各操作有固定耗时，倒计时归零即判负；核心机制不可绕过（如无一卡通不能进图书馆、西楼必须合成锤子才能出门）；所有失败场景要有中文提示，不能静默失败；工程上须通过 Maven 构建、Checkstyle 规范检查和 JUnit 测试，联机模式下计时以服务端为准。

表 2-2 五关流程与通关条件

关卡	开放房间数	单关限时	核心目标	过关条件
第 1 关	5	240 秒	熟悉换卡流程	持一卡通，寝室 sleep
第 2 关	7	300 秒	归寝单 + submit	双证齐全并 submit 后 sleep
第 3 关	9	420 秒	主楼断电 + 西楼解谜	手电、双证、西楼脱困后 sleep
第 4 关	10	540 秒	图书馆领单 + 喂猫支线	换卡、领单、submit 后 sleep
第 5 关	10	720 秒	全图探索 + 密码锁	解锁 20000527、双证 submit 后 sleep

表 2-3 主要操作耗时（节选）

动作	耗时
进入房间 / back 回退	15 秒
look 查看场景	10 秒
take / drop	20 秒 / 10 秒
use / combine / submit	25 秒
NPC 对话	30 秒
eat 食物	25 秒
feed 喂猫	60 秒
无手电闯主楼	额外 60 秒

2.2.2 总体架构设计

我们对比过三种方案：纯 B/S（规则全在前端）、纯桌面（仅 Java 控制台/Swing）、混合架构（Java 内核 + Spring REST + Vue 前端）。最终采用混合架构，理由是：Zuul 天然适合命令模式，规则集中在 Java 内核便于测试；同时又能满足课程对 Spring Boot、RESTful API 和 Vue 的要求。

系统自上而下分为四层。最上层是客户端层：Vue 3 图形前端 vue-portal 是答辩主入口，控制台 Parser/Main 保留为课程原始文本模式。中间是服务层：Spring Boot 提供 /api/solo、/api/game、/api/auth 等 REST 接口，内嵌 Tomcat，默认 8080 端口；GameEngineFacade 把 HTTP 请求转成内核命令，不在 Web 层重复写五关逻辑。再往下

是游戏内核：包路径 `cn.edu.whut.sept.zuul`，不依赖 `Spring`，单机与联机共用同一套 `Game`、`LevelManager`、`CommandManager`。最底层是持久化层：H2 本地文件库 `data/zuul-save`，存关卡号、剩余秒数、用户账号、通关记录等摘要字段，不序列化整个对象图。

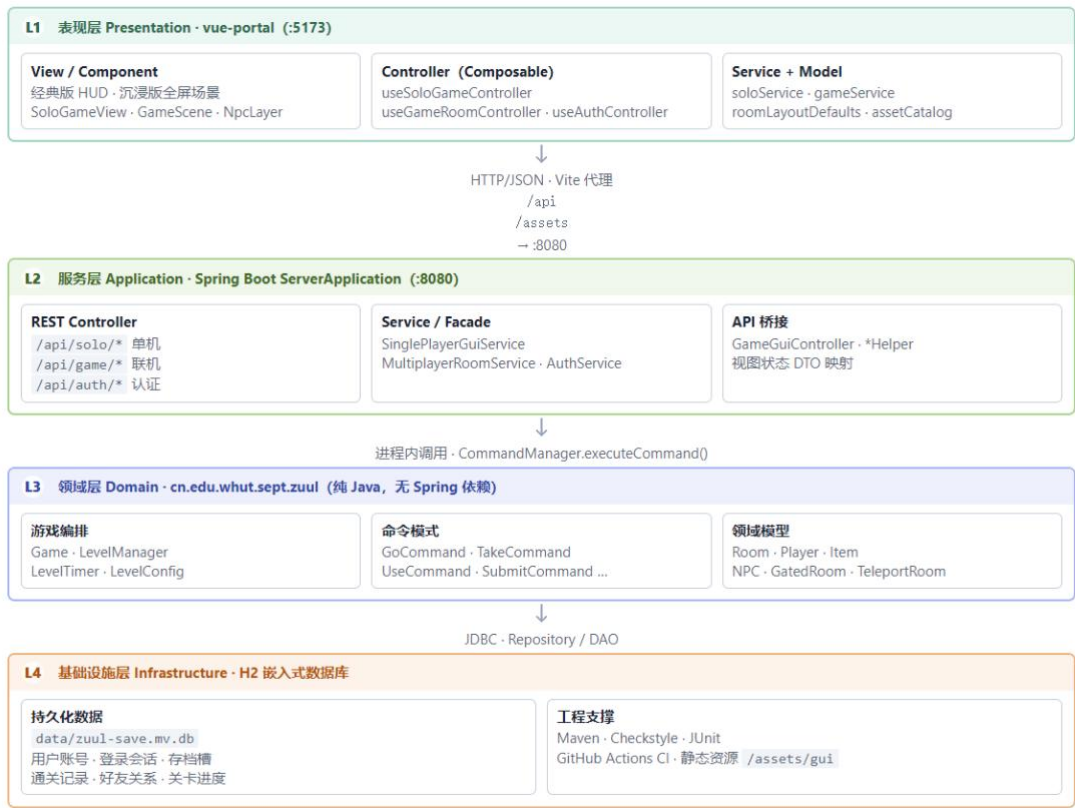


图 2-1 系统架构图

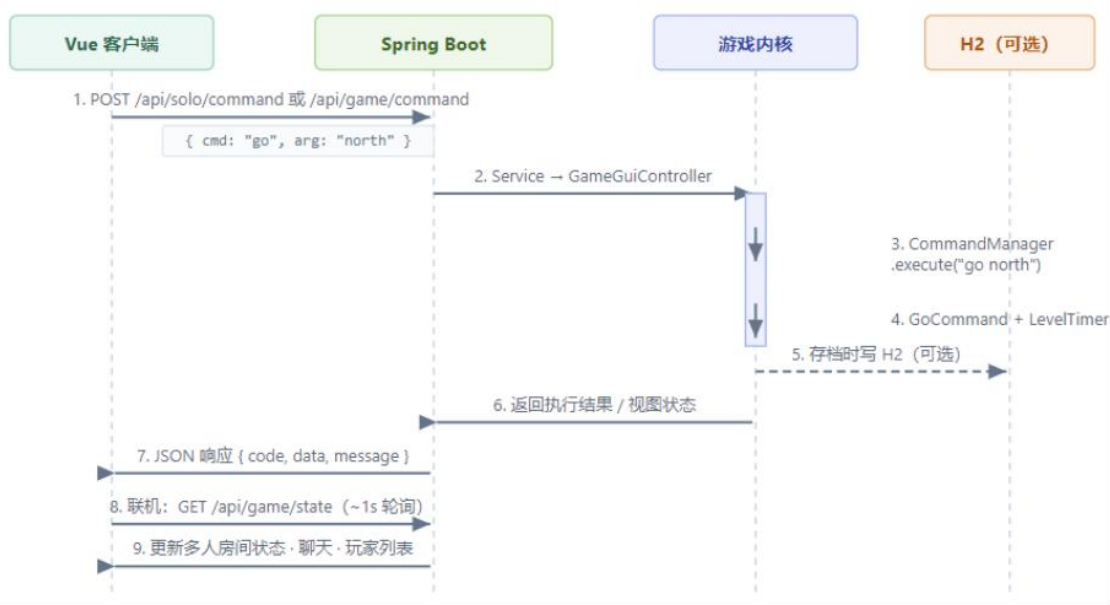


图 2-2 向北走命令时序图

2.2.3 游戏内核分层设计

内核我们按四层组织，和课程讲的分层架构、MVC 思想对应。

表现层负责接收输入和展示：Parser 解析文本命令，Vue 的 View/Component 渲染场景、背包、计时 HUD。应用层负责流程编排：Game 协调各组件，LevelManager 管五关进度与失败重开，LevelTimer 管熄灯倒计时，CommandManager 分发命令；gui/*Helper 为 Vue API 桥接准备视图数据。领域层封装业务对象：Room、Player、Item、NPC，以及 DarkRoom（断电罚时）、GatedRoom（条件门）、TeleportRoom（随机传送）等特殊房间。基础设施层是 H2 DAO，负责存档、用户、好友关系等持久化。

命令扩展我们采用命令模式：use、submit、combine 等各自独立成类，注册到 CommandManager，新增命令不改 Game.play() 的大段分支，符合开闭原则。在我们设计里，每个 Command 类似 Spring MVC 的 Controller：命令名是路由，execute(Game) 是处理逻辑，规则仍在领域层。

2.2.4 地图与房间设计

地图在 Game.createRooms() 中一次构建、全局不变，共十间房：校门、博学主楼/北楼/东楼/西楼、教育超市、寝室、体育馆、越苑食堂、图书馆。关卡差异不由重建地图实现，而由 LevelManager 配合 LevelConfig 控制每关开放哪些出口、刷新哪些物品和公告。这种“固定拓扑 + 按关解锁”的设计，避免了五套地图重复维护，也降低了多人改 Game.java 时的冲突概率。

表 2-4 十房地图与特殊规则

房间 ID	名称	特殊类型	说明
gate	校门	普通	每关起点
boxue_main	博学主楼	DarkRoom	第三、四关断电，无手电罚时退回
boxue_west	博学西楼	困锁	第三关起进入后东侧出口锁定，须锤子出门
library / dormitory	图书馆 / 寝室	GatedRoom	须持一卡通；寝室还需 submit
gymnasium	体育馆	TeleportRoom	第五关启用随机传送
其余	北楼、东楼、超市、食堂	普通	按关配置物品与 NPC

2.2.5 前后端与 API 设计

Vue 前端我们按 model / service / controller / view / component 分层：service/ 封装 REST 调用，controller/ 用 Composable 编排状态，页面包括首页、登录注册、单机选关 (/solo)、单机对局、联机大厅 (/multiplayer)、联机对局等。界面提供经典版和沉浸版两种模式，共用同一套 API。静态资源（房间底图、物品图标、NPC 立绘）放在 src/main/resources/assets/gui/，Java 与 Vue 共用。

Spring 后端按 Controller → Service → Repository 三层组织。单机走 SinglePlayerController (/api/solo/*)，联机走 GameController 和 RoomController (/api/game/*、/api/rooms/*)，认证走 AuthController。联机时服务端持有权威 Game 实例和 LevelTimer，客户端通过命令 API 和状态拉取保持同步，计时以服务端为准。

2.2.6 功能模块划分

我们把全部功能按 F/E/T 三类编号管理，和 GitHub Issue 一一对应。

表 2-5 功能模块总览

类别	编号	内容	我们组的状态
实训基础	F1—F5	look、back、传送、背包、eat	已完成
拓展功能	E1—E17	新命令、五关规则、计时罚时	已实现，持续联调
高级功能	F6—F8	联机、Vue GUI、H2 存档	已实现
测试交付	T1—T7	单元/集成/系统/联机测试与报告	57 个测试类已入库

拓展命令中，use 负责超市换卡、西楼砸锁、图书馆盖章；submit 在寝室门口交归寝单，错物直接失败；combine 在西楼合成锤子；unlock 解体育馆和第五关寝室密码；feed 喂猫换饼干；sleep 是每关最终通关动作。系统模块中，LevelManager 管选关、过关、失败重开；LevelTimer 管熄灯倒计时；ActionTimeCost 管各命令扣秒；E12—E15 分别对应黑暗罚时、条件门、房间解锁和西楼困锁。

2.3. 团队组成与分工

我们小组共四人，推选肖梦琪为组长，负责组织协调、PR 审核和 dev → master 发布。四人共用 GitHub 仓库 wutcst/kai-fa-519club。我们按功能编号和 Issue 分配任务，采用权重分配：肖梦琪与刘晶各约 31%，彭慧星与庞绮君各约 19%，核心开发量集中在前两人，后两人侧重专项模块、联调与文档测试。

肖梦琪承担项目主干与集成，具体包括 LevelManager、LevelTimer 及 E11 操作耗时；扩展 Room 的 E12 黑暗罚时、E13 条件门、E15 按关解锁；F8 H2 框架（表结构、用户认证、存档 API）；Spring Boot REST 与内核 GameGuiController 桥接；F6 联机服务端；T1/T4/T6 测试；全组联调、PR 审核和 CI 发布。

刘晶承担拓展命令与前端，具体包括 E1—E8 全部拓展命令；第 1—3 关配置；E3/E14 西楼合成与砸门；E16 干扰物品；F7 Vue GUI（经典版/沉浸版、单机 /solo、联机 /multiplayer）；拓展命令 JUnit 与 help 文案；T2/T3 测试。

彭慧星负责中段关卡与联机协作，具体包括第 4 关数据；E12—E14 联调；E17 第五关体育馆传送；E5 feed、E6 sleep；第 3—4 关 E16 干扰物配置；T5 系统测试。

庞绮君负责终章关卡与文档测试，具体包括 E15 房间解锁、E7 饼干加时、E14 西楼困锁；第 1—2 关与第 5 关配置；F8 DAO/SQL 与读档；README、架构文档、试玩清单维护；T7 测试报告撰写。

表 2-6 成员分工一览

成员	角色	权重	个人分支	主要负责
肖梦琪	组长	约 31%	feature/xmq	E9—E14 框架、F6/F8、Spring API、集成发布
刘晶	组员	约 31%	feature/lj	E1—E8 命令、F7 Vue GUI、T2/T3
彭慧星	组员	约 19%	feature/phx	E5/E6/E17、第 3—4 关、T5
庞绮君	组员	约 19%	feature/pqj	E15/E7/E14、第 5 关、文档、T7

我们的分工不是“各写各的就不管了”，而是按模块边界划责：肖梦琪先把关卡框架和 Room 规则底层搭好，其他人再往上加命令和关卡配置；刘晶的命令类和 Vue 界面需要和内核 API 对齐；彭慧星、庞绮君按关卡拆分 E16 和房间数据，减少同时改 Game.java 的情况。联调热点（计时扣秒、联机同步、西楼砸锁）由肖梦琪和刘晶结对过读；文档改动统一走 feature/docs 分支，和功能 PR 分开。

表 2-7 Issue 与负责人对应（27 项节选）

GitHub	任务	主责	依赖
#12—#16	E9—E13 关卡与计时	肖梦琪	—
#17	E15 房间解锁	庞绮君	#12
#18—#21、#25	E1—E4、E8 命令	刘晶	#12
#22、#23、#28	E5/E6/E17	彭慧星	#18 等

#29	F6 联机	肖梦琪 + 刘晶	#12—#14
#30	F7 Vue GUI	刘晶 + 肖梦琪	#13、#14、#17
#31	F8 H2	肖梦琪 + 刘晶	#12、#13
#32—#38	T1—T7 测试	全员	功能合并后

2.4. 软件工具与项目约束

我们选用的工具链围绕“Maven 构建 + GitHub 协作 + Spring/Vue 开发 + CI 自动化”组织，和课程五项实践要求（架构、过程管理、分支、评审、CI/CD）一一对应。

表 2-8 工具链与用途

类别	工具	我们组的用法
版本管理	Git + GitHub	代码托管、Issue 跟踪、PR 评审
构建	Maven 3.8+	依赖管理、编译、测试、打包
后端	Spring Boot 2.7	REST API、内嵌 Tomcat
前端	Vue 3 + TypeScript + Vite	图形界面、类型检查
数据库	H2	本地存档 data/zuul-save
规范	Checkstyle	Java 源码格式门禁
测试	JUnit 5	57 个测试类
CI	GitHub Actions	自动 Checkstyle → test → package

强约束我们在 CONTRIBUTING.md 和会议纪要里写死了，也是评分的硬指标：所有 Java 源码必须通过 Checkstyle；合并前 mvn test 必须通过；代码在个人 feature/xxx 写、文档在 feature/docs 写；每个 PR 至少一人 Approve；PR 描述写 Closes #xx；禁止 force push 到 dev/master；交付物包括 README、根目录 REPORT、B 站演示视频和 master 稳定版。

本地门禁我们和 CI 对齐，合并 PR 前必须跑通：

```
mvn checkstyle:check
```

```
mvn test
```

```
cd vue-portal; npm run build
```

3. 小组软件过程

我们以开发小组为整体，采用短周期迭代 + 主干集成（dev）的软件过程，用 GitHub 作为统一管理平台，把需求、代码、评审、CI 和发布串成 DevOps 闭环。下面按仓库、版本计划、分支、规范、测试、发布和 AI 辅助分别说明。

3.1. 代码仓库

我们的代码仓库托管在 GitHub: <https://github.com/wutest/kai-fa-519club>。四人全部加入同一组织/仓库，所有开发成果都在仓库里提交，老师可以直接从 commit 历史、分支、Issue 和 PR 记录核对每人工作量。

仓库主要目录结构如下。Java 游戏内核在 `src/main/java/cn/edu/whut/sept/zuul/`，包含 `command/`（各命令类）、`level/`（关卡与计时）、`gui/`（Vue API 桥接）、`infrastructure/`（H2 持久化与 Spring 服务端）。Vue 前端在 `vue-portal/`，按 `model/service/controller/view/component` 分层。文档在 `docs/`，包括需求说明、架构设计、Issue 清单、会议记录、集成测试场景等。CI 配置在 `.github/workflows/maven.yml`，协作规范在 `CONTRIBUTING.md` 和 PR 模板

我们约定：功能代码走个人 `feature/xxx` 分支，文档走 `feature/docs` 分支，集成都汇入 `dev`，答辩稳定版从 `dev` 合并到 `master`。H2 数据目录 `data/zuul-save` 不随 Git 同步，换电脑需复制 `data/` 或重新注册。

3.2. 软件版本计划

我们把 6 月 1 日至 6 月 20 日划成四个迭代，每个迭代有明确目标和完成标志，对应 README 功能勾选表和 GitHub Issue 的关闭进度。

第一阶段（6/1—6/7）我们目标是搭好关卡框架和命令骨架。肖梦琪交付 E9/E10/E11 与 Room 规则底层，刘晶交付 E1/E2/E6/E8 与第 1—2 关配置，打通「换卡 + sleep」和「归寝单 + submit」两条主线。彭慧星、庞绮君参与评审试玩，整理 Issue 到 GitHub。

第二阶段（6/8—6/14）我们目标是五关单机贯通。刘晶补齐 E3—E8 与其余命令和第 3 关，肖梦琪完成 E12/E13/E15 底层和第 4 关规则，彭慧星交付第 4 关数据，

刘晶启动 vue-portal 场景 UI。本阶段结束时，五关应能通过文本模式或 Vue 界面玩通。

第三阶段（6/15—6/18）我们目标是 F6/F7/F8 联调并冻结功能。肖梦琪主导 H2/认证与全组联调，刘晶完善 Vue 经典/沉浸 UI，彭慧星完成 F6 双人演示，庞绮君完成 F8 读档。6 月 18 日我们功能冻结，之后只修 Bug 和补测试。

第四阶段（6/19—6/20）我们目标是测试闭环和答辩交付。全组跑 T1—T7，庞绮君定稿测试报告，肖梦琪发布 master，全组录制 B 站演示视频，提交 REPORT 电子版。

表 3-1 版本计划与 Issue 对应

版本阶段	时间	对应 Issue	完成标志
v0.1 框架版	6/1—6/7	#12—#18、#23	第 1—2 关可玩
v0.2 功能版	6/8—6/14	#19—#28	五关单机贯通
v0.3 集成版	6/15—6/18	#29—#31	F6/F7/F8 可演示
v1.0 答辩版	6/19—6/20	#32—#38	测试报告 + master 发布

Issue 实施顺序我们按依赖关系排列：先 #12→#13→#14 搭计时框架，再并行 #15—#17 做房间规则，然后 #18—#28 补命令和关卡配置，接着 #30/#31 做 GUI 和存档，最后 #29 做联机（依赖计时框架），功能合并后再做 #32—#38 测试。

3.3. 开发分支模型与提交流程

我们采用精简分支策略：长期只保留 master（答辩发布）和 dev（集成）两个主分支；辅助分支限定四类——每人固定的 feature/姓名拼音、文档用的 feature/docs。我们禁止一人开多个功能分支，也禁止 force push 到 dev/master。

我们的日常提交流程是：每天先从 dev pull 最新代码，merge 进自己的 feature/xx 再开发；功能完成后本地跑通 Checkstyle 和测试，改了 Vue 还要 npm run build；推送后在 GitHub 提 PR 到 dev，至少一人 Review、CI 绿灯后合并；PR 描述里写 Closes #xx 关闭对应 Issue；发布周由组长把 dev 合并到 master 并打 tag。

表 3-2 分支类型与用途

分支	用途	谁可合并到 dev
master	答辩稳定版	仅组长（从 dev）

dev	集成测试	全员 PR + Review
feature/xmq 等	个人日常开发	本人 PR
feature/docs	文档	全员 PR

冲突热点是 Game.java、CommandWords.java、pom.xml。我们的预防办法是按模块分文件、每日 sync dev、小步提交；改热点文件前先在群里说一声。

提交信息我们约定用前缀：feat: 新功能、fix: 修 Bug、test: 补测试、docs: 文档。例如 feat: 实现 submit 归寝单判定、test(T4): 补充集成测试。

3.4. 代码规范与检查流程

我们重视编程规范，把它作为合并门禁的第一道关。Java 代码规范由 Checkstyle 强制执行，配置文件是项目根目录的 checkstyle.xml，在 Maven validate 阶段自动执行，测试代码也纳入检查范围。

我们的 Code Review 关注点不只是格式，还包括：有没有破坏 F1—F5 已有行为；新命令是否在 CommandManager 正确注册；LevelTimer 扣秒是否和 Vue 显示一致；H2 读写有没有覆盖异常；联机命令有没有破坏服务端权威状态。每个 PR 至少一人 Approve 才能合并，PR 模板（.github/PULL_REQUEST_TEMPLATE.md）里有自查清单：Checkstyle 是否通过、JUnit 是否通过、有没有手动试玩、是否影响熄灯计时等。

架构热点（计时器、关卡切换、联机同步）我们采用结对过读：肖梦琪和刘晶联调命令与 LevelManager，彭慧星和肖梦琪对接 F6 协议。Review 评论要具体到文件和行号，作者改完 push 新 commit 直到 Approve。组长对 dev → master 做终审。

本地规范和 CI 完全对齐，合并前必须跑：

```
mvn checkstyle:check    # 规范检查
mvn test                # 单元测试
mvn package             # 打包验证
```

Vue 改动我们另外跑 cd vue-portal; npm run build 做 TypeScript 编译检查。Git Hub Actions 目前覆盖 Java 三步（Checkstyle → JUnit → package），Vue 构建在 PR 合并前本地执行。

3.5. 项目测试计划与实施流程

我们把测试按 T1—T7 七项 Issue 组织，覆盖单元测试、集成测试、系统测试、联

机回归和测试报告，和课程“自动化测试”要求对应。目前仓库里已有 57 个测试类，CI 每次 push/PR 自动跑全部 JUnit 用例。

单元测试（T1—T3） 我们随功能 PR 一并提交。T1 由肖梦琪负责，覆盖 Level Manager、LevelTimer、操作罚时、黑暗罚时、条件门、房间解锁等关卡系统。T2 由刘晶主责、彭慧星和庞绮君协作，覆盖各拓展 Command，每个命令至少一个正向和一个异常用例。T3 由刘晶和肖梦琪负责，覆盖 GUI 桥接类（GameGuiController、*Helper）和 H2 存档（SaveRepository、GamePersistenceService），以及 SinglePlayerApiTest、MultiplayerApiTest 等 API 测试。

集成测试（T4） 由肖梦琪负责，测试类 GameIntegrationTest 串联关卡、计时和命令，至少覆盖三条场景：第 1 关通关、超时失败、第 3 关西楼脱困。我们在 docs/集成测试场景.md 里写了 IT-01 到 IT-06 的步骤和预期。

系统测试（T5） 由彭慧星和庞绮君负责，用黑盒方式跑五关全流程和失败路径（超时、错交归寝单、无一卡通、密码错误、西楼无锤等），分别在控制台文本模式和 Vue GUI（经典/沉浸）下各完成至少第 1 关和第 5 关，记录到系统测试用例表。

联机与回归测试（T6） 由肖梦琪和刘晶负责，验证 F6 至少两人联机、计时以服务端为准、联机改动后单机五关仍可通关、CI 连续绿灯、Actions 制品 JAR 可运行。

测试报告（T7） 由庞绮君主笔，汇总 T1—T6 的用例数、通过率、缺陷列表和 CI 截图，6 月 20 日前定稿。

表 3-3 测试计划一览

编号	类型	主责	覆盖范围	验收标准
T1 #32	单元	肖梦琪	关卡与计时	LevelManager/Timer 关键分支有断言
T2 #33	单元	刘晶等	拓展命令	每 Command ≥ 1 正 +1 反
T3 #34	单元	刘晶/肖梦琪	GUI + H2	API 桥接与存档读写
T4 #35	集成	肖梦琪	跨模块串联	≥ 3 条集成场景
T5 #36	系统	彭/庞	五关黑盒	主路径 + 失败路径
T6 #37	联机回归	肖/刘	F6 + CI	联机演示 + 单机回归
T7 #38	文档	庞绮君	全部	报告定稿

我们的测试实施流程是：功能开发者在个人分支写功能代码的同时补对应单元测试 → PR 合并到 dev 后肖梦琪补集成测试 → 功能冻结后彭慧星、庞绮君跑系统测试

→ 全组联机回归 → 庞绮君汇总写报告。CI 里任何测试失败都会阻断合并，我们不在 CI 红灯时强行合 PR。

3.6. 项目集成与发布规范

我们的集成策略是持续集成到 dev、发布时合并到 master。`.github/workflows/maven.yml` 在 push 到 dev、feature/**或向 master/dev 提 PR 时自动触发，步骤为：Check out → JDK 11 → Checkstyle → JUnit → Maven package → 上传可执行 JAR（Artifact 名 game-jar）和 Surefire 测试报告。任一步失败，PR 不应合并。

演示路径：我们统一用 Spring Boot + Vue：终端 1 跑 `mvn spring-boot:run`（8080），终端 2 跑 `cd vue-portal && npm run dev`（5173），浏览器打开 `http://localhost:5173`。控制台 fat JAR（`java -jar target/*-jar-with-dependencies.jar`）保留为备用入口。

发布规范：我们约定：6 月 18 日功能冻结，之后只修 Bug；6 月 19—20 日全组回归测试；组长发起 dev → master PR，合并后在 master 打 tag v1.0.0；从 Actions 下载 game-jar 作制品备份；同步提交根目录 REPORT.pdf/REPORT.docx、更新 README、上传 B 站视频（标题前缀【武理 26 软工实践】）。

表 3-4 集成与发布检查清单

检查项	命令/方式	通过标准
代码规范	<code>mvn checkstyle:check</code>	零违规
单元测试	<code>mvn test</code>	57 个测试类全绿
打包	<code>mvn package</code>	生成 fat JAR
前端构建	<code>npm run build</code>	TypeScript 编译通过
CI	GitHub Actions	Pipeline 绿灯
功能验收	五关试玩	E1—E17 + F6/F7/F8 可演示
发布	<code>dev → master</code>	tag v1.0.0

读档设计我们有意做了约束：存档只保留关卡号和剩余秒数，读档后回到关卡起点且背包清空，防止玩家反复读档囤道具降低难度。H2 文件库路径 `jdbc:h2:file:./data/zuu1-save`，不随 Git 同步，答辩前我们在演示机本地准备好测试账号和存档数据。

3.7. AI 辅助完成的开发工作

课程允许使用 AI 辅助开发和设计，我们在此说明所用工具及辅助范围。我们主要

使用 Cursor IDE 内置 AI 助手（基于大语言模型）和 GitHub Copilot 类补全，辅助内容集中在文档撰写、测试用例生成、规范修复和代码审查建议，最终架构决策、业务规则和功能实现仍由我们小组讨论确定并人工审核后合入仓库。

文档与报告方面，我们用 AI 辅助起草了 README 功能清单结构、docs/系统架构设计.md 的章节框架、docs/ISSUES.md 的 Issue 正文模板，以及本实训报告的部分段落初稿。这些内容提交前我们都逐段人工核对，确保和实际代码、分工、日期一致，不是照搬 AI 输出。

测试方面，AI 辅助我们生成了部分 JUnit 测试类的骨架和边界用例思路，例如 GameIntegrationTest 的场景步骤、SaveRepositoryTest 的断言结构、GUI 桥接类的 headless 测试模板。测试逻辑和预期结果由对应模块负责人确认，CI 绿灯才算验收通过。

工程维护方面，AI 辅助我们修复过 Checkstyle 报错（如 import 顺序、行宽、Javadoc 缺失），以及 PR 合并前的简单编译错误定位。例如 commit 记录里有 fix: 修复 Checkstyle 与 FriendApiTest 以通过 CI 等，属于 AI 辅助定位、人工确认后提交。

我们未让 AI 独立完成的部分包括：五关策划与数值平衡、LevelManager/LevelTimer 核心逻辑、联机状态同步协议、Vue 界面交互设计、H2 表结构设计。这些由肖梦琪、刘晶等主责成员编写，AI 最多提供语法参考或 API 查文档。

表 3-5 AI 辅助工作对照

辅助领域	使用工具	AI 做了什么	我们做了什么
文档	Cursor	章节框架、Issue 模板初稿	核对功能编号、分工、日期
单元/集成测试	Cursor	测试类骨架、边界用例建议	确断言、跑通 CI
规范修复	Cursor	Checkstyle 报错定位	人工审核 diff 后提交
架构与核心逻辑	—	仅作参考	小组讨论 + 主责人实现
答辩报告	Cursor	段落润色、表格整理	全员 Review 后定稿

我们认同 AI 可以提高文档和测试效率，但软件工程实践的核心——需求分析、架构选型、分工协作、Code Review、CI 门禁——仍由我们四人小组完成。报告中凡涉及 AI 辅助的段落，我们均已人工校对，保证与仓库 wutkst/kai-fa-519club 的实际实现一致。

4. 小组成员任务报告

4.1. 成员一任务分工及实施过程描述

4.1.1 承担任务概述

我是小组组长肖梦琪，权重约 31%，主要负责项目主干架构、关卡系统底层、Spring Boot 服务端、H2 持久化与认证、F6 联机功能，以及集成发布和核心测试。按 GitHub Issue 划分，我主责 #12—#16（E9—E13 关卡与计时框架）、#29（F6 多人联机）、#31（F8 H2 存档与认证），并协作 #30（Vue 与后端 API 桥接）；测试方面主责 #32（T1）、#35（T4）、#37（T6）。日常开发在 feature/xmq 分支进行，文档与报告类改动走 feature/docs。

表 4-1 肖梦琪任务分工一览

类别	编号/Issue	主要内容	我的角色
关卡框架	E9—E14 / #12—#16	LevelManager、LevelTimer、操作罚时、黑暗罚时、条件门	主责
联机	F6 / #29	Spring 联机服务端、房间管理、状态同步	主责
持久化	F8 / #31	H2 表结构、用户认证、存档 API	主责
服务桥接	—	/api/solo、/api/game、/api/auth REST 接口	主责
前端协作	F7 / #30	联机大厅、好友/组队邀请、全局弹窗	主责 + 与刘晶协作
测试	T1/T4/T6 / #32 #35 #37	关卡单元测试、集成测试、联机回归	主责
项目管理	—	PR 审核、dev→master 发布、CI 维护	组长职责

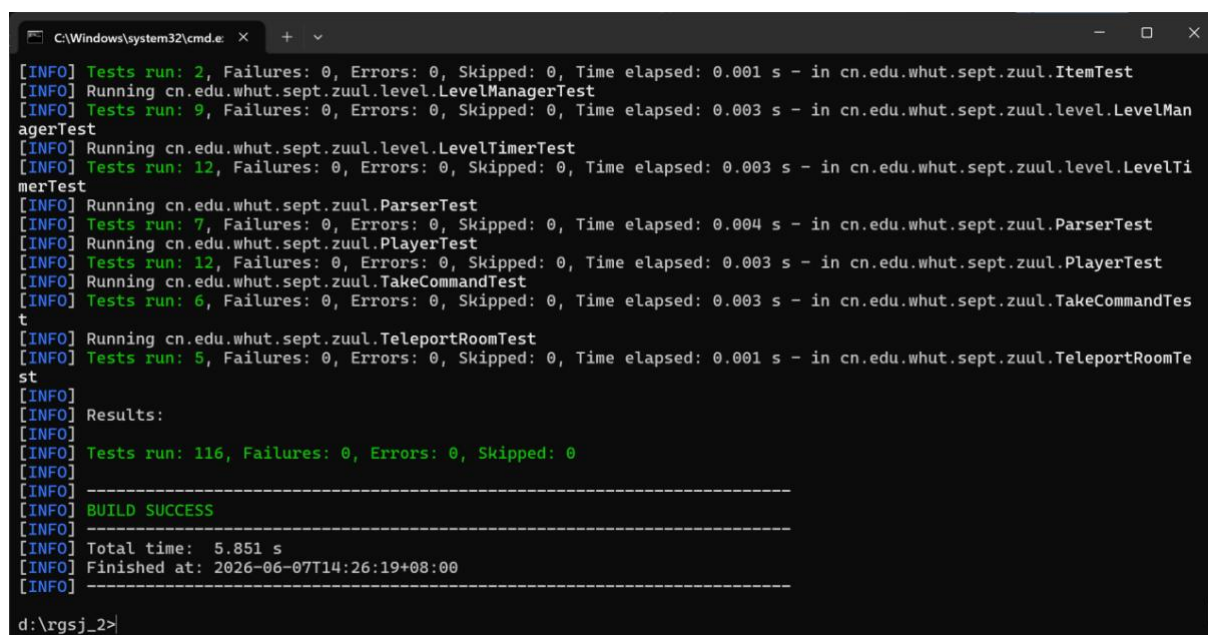
4.1.2 阶段实施过程

（1）第一阶段：关卡与计时框架（6 月 1 日—6 月 7 日）

本阶段我先搭好五关玩法的“骨架”，因为后续所有拓展命令都依赖 LevelManager 和 LevelTimer。我实现了 LevelManager（选关、过关、失败重开、每关清空背包）、LevelTimer（统一显示“距熄灯（23:00）还有 XXX 秒”）、ActionTimeCost（各命令

扣秒），并在 Room 上扩展了 DarkRoom（主楼断电罚时）、GatedRoom（一卡通/ submit 条件门）和按关解锁出口（E15 与 LevelConfig 联动）。

框架搭完后，我补了 LevelManagerTest、LevelTimerTest 等单元测试，并在本地跑 mvn test 验证。6 月 7 日测试全部通过，当时共运行 116 个用例，0 失败，说明关卡与计时核心逻辑已稳定，刘晶、彭慧星、庞绮君可以在此基础上并行开发各自命令和关卡配置。



```
C:\Windows\system32\cmd.e x + v
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s - in cn.edu.whut.sept.zuul.ItemTest
[INFO] Running cn.edu.whut.sept.zuul.level.LevelManagerTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in cn.edu.whut.sept.zuul.level.LevelMan
agerTest
[INFO] Running cn.edu.whut.sept.zuul.level.LevelTimerTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in cn.edu.whut.sept.zuul.level.LevelTi
merTest
[INFO] Running cn.edu.whut.sept.zuul.ParserTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s - in cn.edu.whut.sept.zuul.ParserTest
[INFO] Running cn.edu.whut.sept.zuul.PlayerTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in cn.edu.whut.sept.zuul.PlayerTest
[INFO] Running cn.edu.whut.sept.zuul.TakeCommandTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in cn.edu.whut.sept.zuul.TakeCommandTes
t
[INFO] Running cn.edu.whut.sept.zuul.TeleportRoomTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s - in cn.edu.whut.sept.zuul.TeleportRoomTe
st
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 116, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO]
[INFO] Total time: 5.851 s
[INFO] Finished at: 2026-06-07T14:26:19+08:00
[INFO]
d:\rgsj_2>
```

图 4-1 初期成果图

（2）第二阶段：Spring 服务端与 API 桥接（6 月 8 日—6 月 14 日）

五关单机逻辑基本跑通后，我开始做 Spring Boot 服务端。我新建 ServerApplication，按 Controller → Service → Repository 分层实现 /api/solo/*（单机会话与命令）、/api/game/*（联机命令与状态）、/api/auth/*（注册登录）等接口。核心设计是 GameEngineFacade：REST 层只负责接收 JSON、调用内核 CommandManager，不在 Spring 里重复写五关规则，保证单机、联机、控制台三种入口共用同一套 Java 内核。

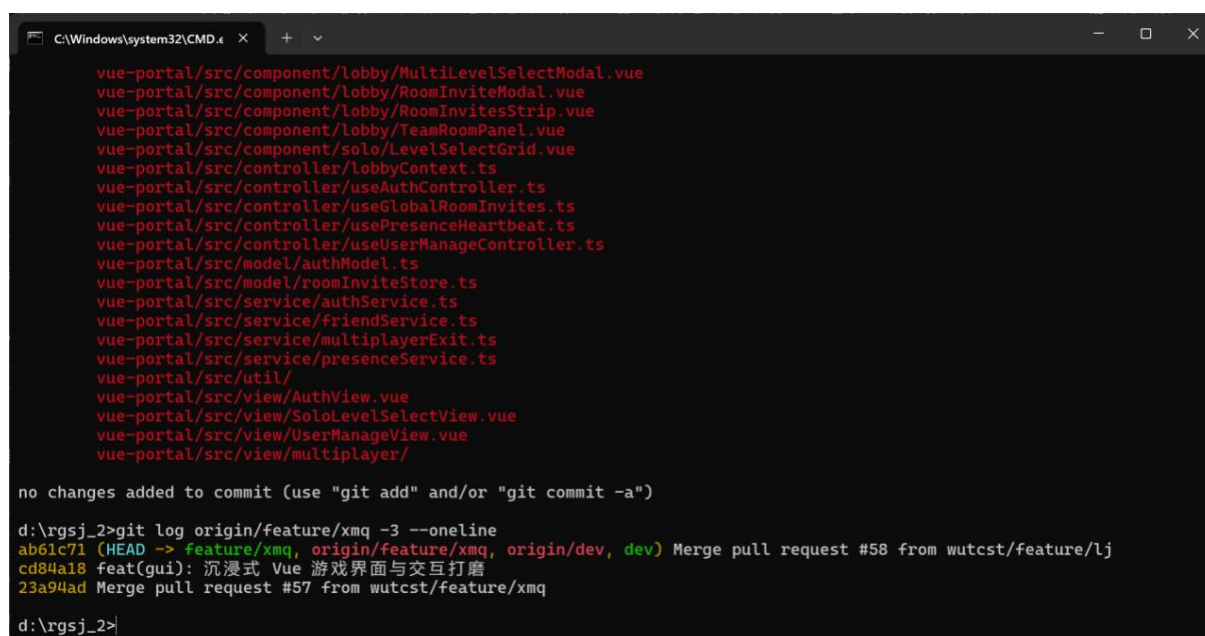
本阶段我还启动了 H2 持久化框架：设计用户表、存档表、通关记录表，实现 SaveRepository、AuthService 等，为 F8 读档和登录打基础。Vue 前端需要通过 REST 驱动游戏，我在 gui/GameGuiController 和各 *Helper 类里做了 API 桥接，把房间描述、背包、计时、结局弹层等状态封装成 DTO 返回给前端。

（3）第三阶段：F6 联机、社交功能与前后端联调（6 月 15 日—6 月 18 日）

联机是我本阶段的重点。我在 Spring 端实现房间创建/加入/退出、多人状态同步、

服务端权威计时（联机倒计时以服务端 LevelTimer 为准），并扩展好友关系、组队邀请、联机大厅等社交能力。前端部分我在 vue-portal 里新增/改动了认证控制器、联机大厅视图、全局邀请弹窗、在线心跳（presence）等模块，使玩家登录后可以创建房间、邀请好友、进入联机对局。

开发完成后，我在 feature/xmq 分支整理提交，执行 `git push origin feature/xmq` 推送到远程，然后在 GitHub 发起 PR #59 合并到 dev。该 PR 标题为“联机社交、全局组队邀请与第三关手电修复”，涵盖好友/组队邀请、联机大厅、Vue 门户与后端联机/单机 API 扩展等内容，是本阶段最主要的一次集成提交。



```
C:\Windows\system32\CMD.exe
vue-portal/src/component/lobby/MultiLevelSelectModal.vue
vue-portal/src/component/lobby/RoomInviteModal.vue
vue-portal/src/component/lobby/RoomInvitesStrip.vue
vue-portal/src/component/lobby/TeamRoomPanel.vue
vue-portal/src/component/solo/LevelSelectGrid.vue
vue-portal/src/controller/lobbyContext.ts
vue-portal/src/controller/useAuthController.ts
vue-portal/src/controller/useGlobalRoomInvites.ts
vue-portal/src/controller/usePresenceHeartbeat.ts
vue-portal/src/controller/useUserManageController.ts
vue-portal/src/model/authModel.ts
vue-portal/src/model/roomInviteStore.ts
vue-portal/src/service/authService.ts
vue-portal/src/service/friendService.ts
vue-portal/src/service/multiplayerExit.ts
vue-portal/src/service/presenceService.ts
vue-portal/src/util/
vue-portal/src/view/AuthView.vue
vue-portal/src/view/SoloLevelSelectView.vue
vue-portal/src/view/UserManageView.vue
vue-portal/src/view/multiplayer/

no changes added to commit (use "git add" and/or "git commit -a")

d:\rgsj_2>git log origin/feature/xmq -3 --oneline
ab61c71 (HEAD -> feature/xmq, origin/feature/xmq, origin/dev, dev) Merge pull request #58 from wutkst/feature/lj
cd84a18 feat(gui): 沉浸式 Vue 游戏界面与交互打磨
23a94ad Merge pull request #57 from wutkst/feature/xmq

d:\rgsj_2>
```

图 4-2 工作成果图

PR 合并前，我按组内规范在本地跑门禁：`mvn checkstyle:check` 显示 0 Checkstyle violations，148 个源文件编译通过；随后跑 `mvn test` 确认联机改动没有破坏已有单机逻辑。PR 需至少一人 Approve 且 CI 通过后才能合入 dev，我作为组长也负责 Review 其他组员 PR，保证集成质量。

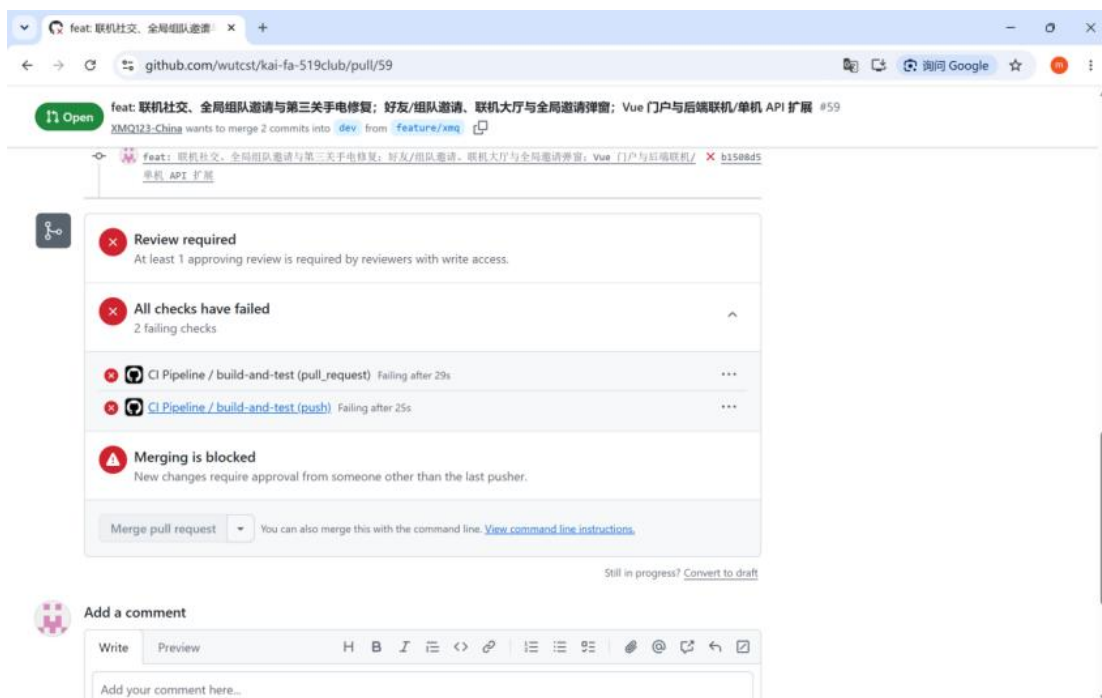


图 4-3 工作图：不通过重新修改

(4) 第四阶段：测试补全、回归与发布准备（6 月 19 日—6 月 20 日）

功能冻结后，我主导 T1/T4/T6 测试补全与 CI 回归。T1 方面补充关卡切换、超时判负、条件门拦截等单元测试；T4 方面编写 `GameIntegrationTest`，覆盖第 1 关通关、超时失败、第 3 关西楼脱困等跨模块场景，并在 `docs/集成测试场景.md` 记录 IT-01—IT-06；T6 方面做联机双人演示和单机五关回归，确认 F6 改动后 F1—F5 与 E 系列仍可用。

```
C:\Windows\system32\cmd.e X + v
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s - in cn.edu.whut.sept.zuul.SubmitCommand
Test
[INFO] Running cn.edu.whut.sept.zuul.TakeCommandTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s - in cn.edu.whut.sept.zuul.TakeCommandTes
t
[INFO] Running cn.edu.whut.sept.zuul.TeleportRoomTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s - in cn.edu.whut.sept.zuul.TeleportRoomTe
st
[INFO] Running cn.edu.whut.sept.zuul.UnlockCommandTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s - in cn.edu.whut.sept.zuul.UnlockCommand
Test
[INFO] Running cn.edu.whut.sept.zuul.UseCommandTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s - in cn.edu.whut.sept.zuul.UseCommandTes
t
[INFO] Running cn.edu.whut.sept.zuul.WestBuildingLockTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s - in cn.edu.whut.sept.zuul.WestBuildingLo
ckTest
[INFO] Results:
[INFO]
[INFO] Tests run: 318, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 17.090 s
[INFO] Finished at: 2026-06-19T11:56:22+08:00
[INFO]
d:\rgsj_2>
```

图 4-4 进行测试工作图

6 月 19 日全量测试结果为 318 个用例全部通过，0 失败，相比 6 月 7 日的 11

6 个用例，测试覆盖面随功能迭代显著扩大，说明我们在开发过程中持续补测试而不是最后突击。

发布前我协调全组在 dev 上完成最后一轮联调，计划由我将验收通过的 dev 合并到 master 并打 tag v1.0.0，作为答辩演示版本。

4.1.3 任务实施过程详细描述

任务一：E9 LevelManager 五关进度管理

任务说明：实现 LevelManager，统一管理五关的选关、通关进下一关、超时/错误失败重开、每关开局清空背包与重置位置。LevelManager 是 E10 计时、E13 条件门、E15 房间解锁、E6 sleep 过关等全部后续功能的前置依赖，必须最先完成。

实施过程：

实施思路：五关差异不重建地图，而是在 LevelConfig 里配置每关限时、开放房间集合和任务提示；LevelManager 持有当前关卡号、最高解锁关、关卡状态（进行中/已通关/已失败/全通），以及西楼困锁、归寝单提交、密码解锁等布尔标志。startLevel(n) 时校验解锁、清空背包、重置玩家到校门、调用 LevelTimer.resetForLevel() 并刷新房间物品。

实验操作：

LevelManager.startLevel(levelNumber):

```
if levelNumber 越界 → 抛 IllegalArgumentException
if levelNumber > highestUnlockedLevel → 抛"尚未解锁"
currentLevel = levelNumber
currentConfig = LevelConfig.forLevel(levelNumber)
state = IN_PROGRESS
重置 dormitorySubmit、westBuildingLock 等全部布尔标志
game.clearAllInventories()
game.resetPlayerPosition(gate)
levelTimer.resetForLevel(currentConfig)
game.applyLevelRoomState(currentConfig) // 刷新物品与公告
打印关卡标题 + 熄灯倒计时文案
```

LevelManager.completeCurrentLevel():

if state != IN_PROGRESS → 拒绝

if currentLevel == 5 → state = GAME_WON, 触发 onAllLevelsCompleted()

else → highestUnlockedLevel++, 自动 startLevel(currentLevel + 1)

LevelManager.failCurrentLevel():

state = FAILED, 提示"本关失败, 输入 restart 重开"

实验中遇到的问题及解决方法:

问题一: 通关后自动 startLevel(nextLevel) 时, 上一关的西楼困锁、submit 完成等标志被带到下一关, 导致第 4 关仍显示“西楼已解锁”。

解决: 在 startLevel() 开头统一重置全部布尔标志 (dormitorySubmitCompleted、westBuildingExitLocked 等), 保证每关状态干净。

问题二: 联机模式下多人共用同一个 Game, 某玩家过关后 clearAllInventories() 会清空所有人背包。

解决: 在 Game 里区分单机/联机会话, 联机过关时只清空当前操作玩家背包, 并在 GameEngineFacade 里用房间锁 synchronized(room.getLock()) 串行化命令。

任务二: E10 LevelTimer 熄灯倒计时

任务说明: 实现 LevelTimer, 全局统一显示「距熄灯 (23:00) 还有 XXX 秒」; 倒计时归零触发 LevelManager.failCurrentLevel()。联机模式下计时以服务端为准, 客户端只接收权威秒数, 不能本地各算各的。

实施过程

实施思路: LevelTimer 绑定 Game, 用 synchronized(lock) 保证扣秒线程安全。单机控制台 play() 时启用 ScheduledExecutorService 每秒自动流逝 1 秒; 单元测试默认关闭 autoTick, 改用手动 deduct()。联机时通过 TimerAuthority 枚举区分三种模式: LOCAL_CLIENT (单机)、SERVER_HOST (服务端 tick)、SERVER_CLIENT (只接收 applyAuthoritativeRemainingSeconds(), 不本地 tick)。

实验操作:

LevelTimer.deduct(seconds):

synchronized(lock):

if !isActive() → return false

```
remainingSeconds = max(0, remainingSeconds - seconds)
```

```
打印 getDisplayText() // "距熄灯 (23:00) 还有 XXX 秒"
```

```
if remainingSeconds == 0 → handleTimeUp()
```

```
→ 打印"熄灯了！本关失败。"
```

```
→ levelManager.failCurrentLevel()
```

```
→ 停止后台 ticker
```

```
LevelTimer.applyAuthoritativeRemainingSeconds(seconds): // F6 联机客户端
```

```
synchronized(lock):
```

```
remainingSeconds = max(0, seconds)
```

```
if remainingSeconds == 0 → handleTimeUp()
```

```
LevelTimer.setTimerAuthority(SERVER_CLIENT):
```

```
停止本地 ticker，后续只接受服务端推送
```

实验中遇到的问题及解决方法：

问题一：控制台模式和 Vue REST 模式同时跑时，后台 ticker 线程在 Game 销毁后仍在扣秒，导致测试间歇性失败。

解决：新增 shutdown() 方法，在退出游戏和 @AfterEach 测试清理里调用，stopTickerInternal() 取消 ScheduledFuture 并关闭线程池。

问题二：联机客户端本地 tick 与服务端 tick 双扣，玩家看到的倒计时比服务端少几十秒。

解决：联机客户端连接成功后立即 setTimerAuthority(SERVER_CLIENT)；服务端房间创建时 setTimerAuthority(SERVER_HOST)；Vue 每次拉 /api/game/state 时用返回的 remainingSeconds 刷新界面。

任务三：E11 操作耗时扣秒

任务说明：实现 ActionTimeCost 常量类，规定 go/look/take/drop/use/submit 等命令成功后的耗时，并在各 Command 成功分支调用 ActionTimeCost.deduct(game, seconds)。只有命令成功执行才扣时，无效操作不扣时。

实施过程：

实施思路：把耗时秒数集中定义在 ActionTimeCost 里，避免各 Command 硬编码

魔法数字。常规命令在 GoCommand、LookCommand 等成功分支末尾扣时；黑暗罚时、喂猫、密码错误等额外罚时也在此类中定义常量，由对应逻辑单独调用。

实验操作：

ActionTimeCost 常量：

GO=15, LOOK=10, TAKE=20, DROP=10

USE/SUBMIT/COMBINE=25, NPC=30

DARK_PENALTY=60, FEED=60, EAT=25

COOKIE_BONUS=300

GoCommand.execute():

if setCurrentRoom(nextRoom) 成功:

 ActionTimeCost.deduct(game, GO)

 printLocationInfo()

Game.setCurrentRoom() 黑暗拦截分支:

if DarkRoom 且无手电:

 打印 PENALTY_MESSAGE

 ActionTimeCost.deduct(game, DARK_PENALTY) // 额外 60 秒

 return false // 不进入房间，也不扣 GO 的 15 秒

实验中遇到的问题及解决方法：

问题一： 黑暗罚时和 go 扣时叠加，无手电闯主楼一次扣 75 秒，和策划“罚 1 分钟”不一致。

解决： 明确规则：黑暗拦截时只扣 DARK_PENALTY（60 秒），不进入房间因此不扣 GO 的 15 秒；在 ActionTimeCostCommandTest 里写断言验证。

问题二： 联机模式下某玩家 look 扣时后，其他玩家 HUD 未刷新。

解决： 命令执行完毕后 GameStateSnapshot.from(game, playerId) 把最新 remainingSeconds 放进 JSON，Vue 轮询 state 接口更新。

任务四： E12 博学主楼黑暗罚时

任务说明： 第三、四关博学主楼断电，玩家无手电筒进入时罚 1 分钟并退回，提示「灯坏了，黑暗中摸索一分钟一无所获」。第五关主楼恢复供电，不再触发黑暗规则。

实施过程

实施思路：新建 `DarkRoom` extends `Room`，提供 `canEnter(Player)` 检查背包是否有「手电筒」。在 `Game.setCurrentRoom()` 里，当目标房间是 `DarkRoom` 且 `LevelConfig.isMainBuildingDark()` 为 `true` 时，先走黑暗准入逻辑；联机下增加 `mainBuildingIlluminated` 标志——任一玩家持手电进入主楼后，全队可通行。

实验操作：

`DarkRoom.canEnter(player):`

```
return player.findItemInInventory("手电筒") != null
```

`Game.canEnterDarkRoom(darkRoom):`

```
if mainBuildingIlluminated → return true    // 联机共享照亮
```

```
return darkRoom.canEnter(getPlayer())
```

`Game.setCurrentRoom(targetRoom):`

```
if targetRoom instanceof DarkRoom && config.isMainBuildingDark():
```

```
    if !canEnterDarkRoom():
```

```
        打印 DarkRoom.PENALTY_MESSAGE
```

```
        ActionTimeCost.deduct(DARK_PENALTY)
```

```
        return false    // 留在原房间
```

```
...正常进入...
```

```
if 进入 boxue_main && isMainBuildingDark():
```

```
    mainBuildingIlluminated = true
```

实验中遇到的问题及解决方法：

问题一： PR #59 联调时，第三关有人带手电进主楼，另一人仍被黑暗罚时。

解决： 联机命令在 `GameEngineFacade` 房间锁内执行，进入主楼时设置 `mainBuildingIlluminated = true`，并在 `GameStateSnapshot` 里同步该标志；Vue 侧根据状态不再对后续玩家弹黑暗罚时。

问题二： 第五关误触发黑暗罚时。

解决： 在 `LevelConfig` 里只有第 3、4 关 `isMainBuildingDark()` 返回 `true`，第 5 关返回 `false`，黑暗分支根本不进入。

任务五：E13 条件门（一卡通与 submit）

任务说明： 图书馆、寝室须持一卡通；第二关起寝室还须先完成归寝单 submit。
不满足条件时拦截进入并给出中文提示（「请刷一卡通」「请先提交归寝单」）。

实施过程

实施思路： 新建 GatedRoom extends Room，用枚举 AccessRule 区分图书馆和寝室两种规则。getDenialMessage(Game) 按顺序检查：是否有一卡通 → 当前关是否要求 submit → submit 是否已完成；返回 null 表示允许进入。

实验操作：

GatedRoom.getDenialMessage(game):

```
if !hasCampusCard(player):  
    if accessRule == LIBRARY → return "未拿到一卡通，无法进入图书馆."  
    else → return "请刷一卡通"  
if accessRule == DORMITORY && config.requiresDormitorySubmit():  
    if !levelManager.isDormitorySubmitCompleted():  
        return "请先提交归寝单"  
return null    // 允许进入
```

Game.setCurrentRoom(targetRoom):

```
if targetRoom instanceof GatedRoom:  
    denial = gatedRoom.getDenialMessage(this)  
    if denial != null:  
        打印 denial  
    return false
```

实验中遇到的问题及解决方法：

问题一： 第一关寝室不需要 submit，但玩家被误拦。

解决： LevelConfig.requiresDormitorySubmit() 只在第 2 关及以后返回 true，第一关 getDenialMessage 只检查一卡通。

问题二： Vue 联机模式下条件门拦截只打印控制台，前端无弹窗。

解决： 在 SinglePlayerGuiService 和 MultiplayerViewMapper 里解析 denial 文案，设置 popupMessage 字段返回给 Vue 弹层显示。

任务六：E15 按关解锁房间出口（与庞绮君协作）

任务说明：十房地图固定，按关逐步开放：第 1 关 5 房 → 第 5 关全图。未开放出口点击时提示「夜色中，这个方向暂未开放。」我负责在 `Game.isRoomAccessible()` 和 `GoCommand` 里接入 `LevelConfig.isRoomUnlocked()` 判定逻辑；庞绮君负责各关 `LevelRoomContent` 物品与公告配置。

实施过程

实施思路：`LevelConfig` 为每关维护 `unlockedRoomIds` 集合；`Game.isRoomAccessible(room)` 查当前关配置；`GoCommand` 在 `setCurrentRoom` 前先检查，未解锁直接打印 `LOCKED_EXIT_MESSAGE` 且扣 `go` 耗时。

实验操作：

`Game.isRoomAccessible(targetRoom):`

```
return levelManager.getCurrentLevelConfig()
    .isRoomUnlocked(targetRoom.getRoomId())
```

`GoCommand.execute():`

```
nextRoom = currentRoom.getExit(direction)
if !game.isRoomAccessible(nextRoom):
    打印 LevelConfig.LOCKED_EXIT_MESSAGE
else if game.setCurrentRoom(nextRoom):
    ActionTimeCost.deduct(GO)
```

实验结果：`LevelConfigRoomUnlockTest`、`RoomUnlockTest` 通过。第 1 关无法直接去体育馆，第 2 关起逐步开放。

任务七：F6 网络多人联机

任务说明：实现 `Spring Boot` 联机服务端：房间创建/加入/退出、多人命令执行、状态同步、好友/组队邀请、联机大厅；计时以服务端 `LevelTimer` 为准。`Vue /multiplayer` 前端经 `REST` 驱动，不破坏单机流程。

实施过程

实施思路：内核层已有 `GameEngineFacade` 和 `GameRoom`，我在 `Spring` 层封装

MultiplayerRoomService, 用 GameRoomRegistry 管理内存房间, UserPresenceRegistry 跟踪在线状态, RoomInviteRegistry 管理邀请。每个房间持有一个权威 Game 实例; 命令经 POST /api/game/command 进入 Facade, 在房间锁内执行并返回 GameStateSnapshot JSON。

实验操作:

MultiplayerRoomService.createRoom(roomName, hostName, userId):

if userId 已在某房间 → 拒绝

room = roomRegistry.createRoom(...)

room.getGame().getLevelTimer().setTimerAuthority(SERVER_HOST)

syncRoomPresence(room)

return toSessionDto(room)

GameEngineFacade.executeCommand(room, playerId, cmd, arg):

synchronized(room.getLock()):

game.setActiveOnlinePlayer(playerId)

OutputCapture 捕获控制台输出

quit = commandManager.executeCommand(cmd, arg, game)

state = GameStateSnapshot.from(game, playerId)

return GameCommandResult(output, quit, state)

Vue 联机流程:

POST /api/rooms → 创建房间

POST /api/rooms/{id}/join → 加入

POST /api/game/command → 发命令

GET /api/game/state → 拉取剩余秒数与房间描述

实验中遇到的问题及解决方法:

问题一: 两人同时 go 产生竞态, 一人状态覆盖另一人。

解决: 所有联机命令在 synchronized(room.getLock()) 内执行; 状态快照按 playerId 区分各玩家视角。

问题二: PR #59 合并时 CI 报 FriendApiTest 失败。

解决: 修复好友 API 边界用例和 Checkstyle import 顺序, 重新 push 后 CI 通

过（见 commit f179b96）。

任务八：F8 H2 存档与用户认证

任务说明：用 H2 本地文件库 data/zuul-save 实现用户注册登录、游戏存档读档、通关记录。存档只存关卡号、剩余秒数、进度标志和背包摘要，读档后回到关卡起点且背包清空（与需求一致）。

实施过程

实施思路：H2Database 初始化表结构；AuthService 管注册/登录/Token 校验；SaveRepository 用 PreparedStatement 读写 game_save 表，关联 SaveItemRepository 存背包物品。Spring AuthController 暴露 /api/auth/*，Vue 登录页携带 X-Auth-Token 请求头。

实验操作：

SaveRepository.insertSnapshot(snapshot):

```
INSERT INTO game_save (player_name, level_number, remaining_seconds,  
    current_room_id, dormitory_submit, west_exit_locked, ...)  
saveItemRepository.replaceItemsForSave(saveId, inventory)
```

AuthService.signIn(username, password):

```
user = userRepository.findByUsername(username)  
if PasswordHasher.verify(password, user.hash) → 创建 Session, 返回 token
```

读档流程：

```
GET /api/saves/{id} → 加载快照  
levelManager.startLevel(snapshot.levelNumber)  
levelTimer.reset(snapshot.remainingSeconds)  
恢复 progress 布尔标志，清空背包（按需求设计）
```

实验中遇到的问题及解决方法：

问题一：H2 文件库路径相对启动目录，Spring Boot 和单元测试工作目录不同，测试污染生产数据。

解决：测试用内存库或 @TempDir 临时文件；SaveRepositoryTest 用独立 H2 实例。

任务九：Spring REST API 桥接（单机 + 联机）

任务说明：实现 `SinglePlayerController (/api/solo/*)`、`GameController (/api/game/*)`、`RoomController`、`AuthController`，让 Vue 经 REST 驱动内核，不在 Spring 层重复五关规则。

实施过程

实施思路： `SinglePlayerGuiService` 维护 `SinglePlayerSessionRegistry` (`sessionId` → `Game` 实例)；每个 REST 请求找到对应 `Game`，调 `GameGuiController.execute()` 或 `Facade`，把控制台输出和视图 DTO 封装成 JSON 返回。统一响应格式 { `code`, `data`, `message` }。

实验操作：

POST `/api/solo/sessions` → 创建 `Game`，返回 `sessionId`

POST `/api/solo/sessions/{id}/command` { `commandWord`, `secondWord` }

→ `singlePlayerGuiService.executeCommand()`

→ 返回 `SoloCommandResponseDto`(`output`, `remainingSeconds`, `popupMessage`, ...)

GET `/api/solo/sessions/{id}/state` → `SoloViewStateDto`（房间描述、背包、出口、计时）

任务十：T1/T4/T6 测试与 CI 回归

任务说明：主责 T1 关卡单元测试、T4 集成测试、T6 联机回归；维护 CI 流水线，保证 `push/PR` 自动跑 `Checkstyle` → `JUnit` → `package`。

实施过程

实施思路：T1 随 E9—E16 功能 PR 提交 `*Test.java`；T4 编写 `GameIntegrationTest`，覆盖 IT-01—IT-06 六条跨模块场景，文档写在 `docs/集成测试场景.md`；T6 在 F6 合并后跑联机 + 单机回归 + CI 绿灯验证。

实验操作：

本地门禁（合并 PR 前）：

`mvn checkstyle:check` → 0 violations

`mvn test` → 全部用例通过

mvn package → 生成 fat JAR

cd vue-portal; npm run build

CI (.github/workflows/maven.yml):

push/PR → Checkout → JDK 11 → checkstyle → test → package → 上传 game-jar

实验中遇到的问题及解决方法:

问题一: 6 月初只有 116 个用例, F6/F8 合入后多个 API 测试失败。

解决: 逐类修复 FriendApiTest、Checkstyle import 顺序, 联调期间持续补测试, 6 月 19 日增至 318 个用例全绿。

4.1.4 实施成果

(1) 代码与功能成果

我负责的核心模块已合入 dev 分支, 主要包括:

关卡系统: LevelManager、LevelTimer、LevelConfig、ActionTimeCost, 支撑五关进度、熄灯倒计时和操作罚时;

房间规则: DarkRoom、GatedRoom、西楼困锁状态、按关解锁出口;

Spring 服务端: ServerApplication、SinglePlayerController、GameController、Room Controller、AuthController 及对应 Service/Repository;

联机与社交: 房间管理、多人状态同步、好友/组队邀请、联机大厅 API 与前端联调;

H2 持久化: 用户注册登录、存档读档、通关记录写入;

测试: 关卡/计时单元测试、集成测试 GameIntegrationTest、联机 API 测试等, 推动全项目测试用例从 116 增至 318。

(2) 界面与演示成果

联调完成后, Vue 首页已支持账号登录、单人闯关入口、联机大厅入口, 以及经典版/沉浸版场景模式切换。下图是 6 月 20 日答辩前本地运行的首页效果: 左上角显示已登录用户, 中间可选择“开始单人游戏”或“进入联机大厅”, 底部可切换经典/沉浸模式, 说明我负责的认证、联机和 API 桥接已在前端完整打通。



图 4-5 前端界面图

(3) 工程与协作成果

作为组长，我除了写代码，还承担了以下协作工作：

在 GitHub 创建并维护 #12—#38 共 27 个 Issue，明确负责人和依赖关系；

审核组员 PR，处理 Game.java、pom.xml 等冲突热点；

维护 .github/workflows/maven.yml CI 流水线，保证 push/PR 自动跑 Checkstyle
→ JUnit → package；

组织 6 月 3 日功能分工会议，输出会议记录和 Issue 实施顺序。

表 4-2 肖梦琪主要实施成果汇总

成果类型	具体内容	验证方式
关卡框架	E9—E13 全部落地	LevelManagerTest、LevelTimerTest 通过
联机功能	F6 房间、同步、社交	PR #59、MultiplayerApiTest、双人演示
持久化	F8 认证与存档	SaveRepositoryTest、AuthServiceTest
测试	T1/T4/T6	318 用例全绿
规范	Checkstyle 零违规	图 4-5
集成	dev 分支可构建可演示	CI Pipeline 绿灯、图 4-5 首页

4.2. 成员二任务分工及实施过程描述

4.2.1 承担任务总览

成员姓名：刘晶

承担任务总览： 本人负责拓展命令 E1—E4、E8 共五项核心游戏命令的后端实现，以及 F7 Vue 图形界面（主责）、F8 H2 存档（DAO 协作）、F6 联机模式 Vue 前端（协作），同时主责 T2 拓展命令单元测试与 T3 GUI/存档单元测试，并协作参与 T6 联机回归测试。

表 4-3 任务总览表

类别	编号	任务名称	角色
功能开发	E1（#18）	use 命令	主责
功能开发	E2（#19）	submit 命令	主责
功能开发	E3（#20）	combine 命令	主责
功能开发	E4（#21）	unlock 命令	主责
功能开发	E8（#25）	NPC 对话（talk）	主责
功能开发	F7（#30）	Vue 图形界面	主责
功能开发	F8（#31）	H2 数据库存档	协作（DAO 层）
功能开发	F6（#29）	联机模式 Vue 前端	协作
测试	T2（#33）	拓展命令单元测试	主责
测试	T3（#34）	GUI 与存档单元测试	主责
测试	T6（#37）	联机回归测试	协作

4.2.2 任务开发流程

本人的每项任务均按照以下流程完成，从 GitHub 领取任务到最终合并进 dev 分支。

1. 在 GitHub 认领 Issue

在 GitHub 仓库的 Issues 页面找到分配给本人的 Issue，将自己设为 Assignee，确认任务的功能说明、验收标准和依赖关系，待依赖的上游任务合并进 dev 后方可开始开发。

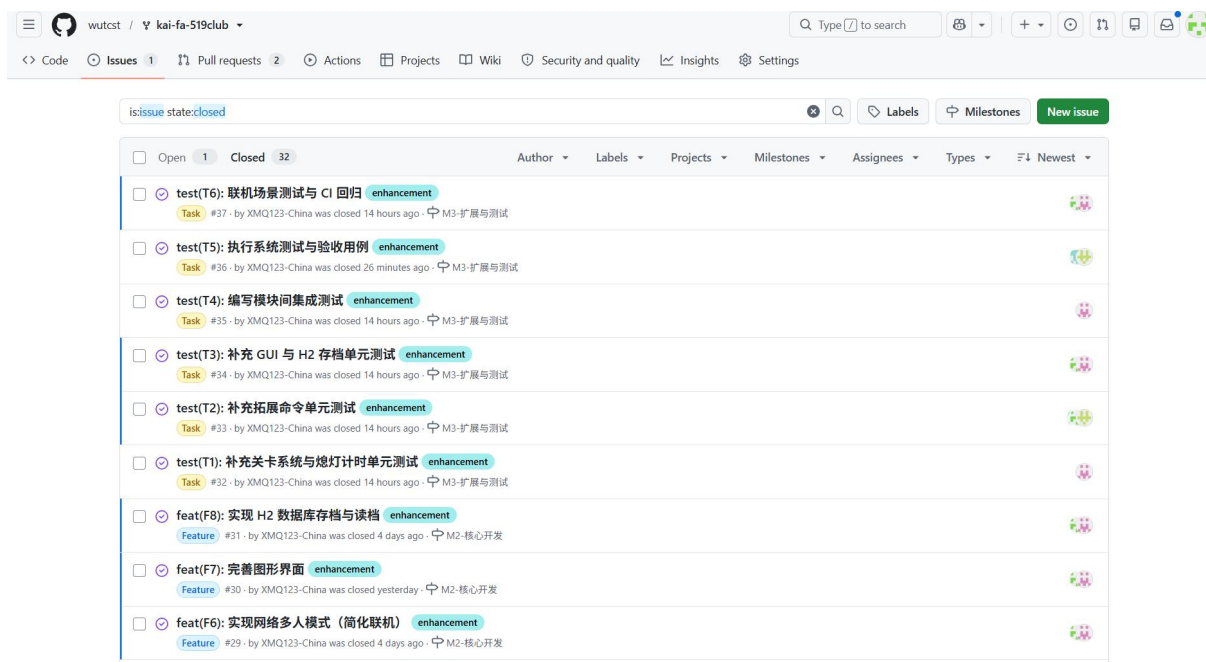


图 4-4: GitHub Issue 页面，显示 Assignee 和验收标准

2. 在 feature/1j 分支开始开发

在本地 feature/1j 分支上进行功能开发，包括新建/修改 Java 源文件、编写对应的 JUnit 测试类，以及根据需要调整 Vue 前端代码。开发过程中随时在本地运行 `mvn checkstyle:check` 和 `mvn test` 验证代码格式和测试是否通过。

```
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s - in cn.edu.whut.sept.zuul.regression.CiRegres
[INFO] Running cn.edu.whut.sept.zuul.RoomUnlockTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s - in cn.edu.whut.sept.zuul.RoomUnlockTest
[INFO] Running cn.edu.whut.sept.zuul.SleepCommandTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.054 s - in cn.edu.whut.sept.zuul.SleepCommandTest
[INFO] Running cn.edu.whut.sept.zuul.SubmitCommandTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s - in cn.edu.whut.sept.zuul.SubmitCommandTest
[INFO] Running cn.edu.whut.sept.zuul.TakeCommandTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s - in cn.edu.whut.sept.zuul.TakeCommandTest
[INFO] Running cn.edu.whut.sept.zuul.TeleportRoomTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s - in cn.edu.whut.sept.zuul.TeleportRoomTest
[INFO] Running cn.edu.whut.sept.zuul.UnlockCommandTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s - in cn.edu.whut.sept.zuul.UnlockCommandTest
[INFO] Running cn.edu.whut.sept.zuul.UseCommandTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in cn.edu.whut.sept.zuul.UseCommandTest
[INFO] Running cn.edu.whut.sept.zuul.WestBuildingLockTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in cn.edu.whut.sept.zuul.WestBuildingLockTes
[INFO] Results:
[INFO] Tests run: 340, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 21.640 s
[INFO] Finished at: 2026-06-20T03:35:27+08:00
[INFO]
```

图 4-5 本地 mvn test 运行通过结果

3. 查看改动状态

功能开发完成后执行 `git status`，确认本次改动涉及的文件，检查是否有遗漏或不该提交的文件（如 `.tools/` 目录）。

```
D:\softwaretest\kai-fa-519club>git status
On branch feature/lj
Your branch is ahead of 'origin/feature/lj' by 7 commits.
(use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .tools/
    "docs/\347\263\273\347\273\237\346\265\213\350\257\225\347\224\250\344\276\213.md"
    src/test/java/cn/edu/whut/sept/zuul/integration/T5SystemPathTest.java

nothing added to commit but untracked files present (use "git add" to track)
```

图 4-6: `git status` 输出

4. 暂存改动文件

使用 `git add` 将本次任务相关的改动加入暂存区，只添加涉及的目录（如 `src/`、`docs/`、`vue-portal/` 等），不提交本地工具目录。

```
D:\softwaretest\kai-fa-519club>git add docs/ src/

D:\softwaretest\kai-fa-519club>git commit -m "test(T5): 执行系统测试与验收用例"
[feature/lj c098f97] test(T5): 执行系统测试与验收用例
2 files changed, 390 insertions(+)
create mode 100644 "docs/\347\263\273\347\273\237\346\265\213\350\257\225\347\224\250\344\276\213.md"
create mode 100644 src/test/java/cn/edu/whut/sept/zuul/integration/T5SystemPathTest.java
```

图 4-7: `git add` 命令执行

5. 提交并填写提交信息

执行 `git commit`，提交信息按项目约定格式填写：类型前缀（`feat / fix / test`）+ 功能编号 + 简要描述，并在 PR 描述中关联对应 Issue 编号（Closes #xx），合并后 Issue 自动关闭。

```
D:\softwaretest\kai-fa-519club>git commit -m "test(T5): 执行系统测试与验收用例"
[feature/lj c098f97] test(T5): 执行系统测试与验收用例
2 files changed, 390 insertions(+)
create mode 100644 "docs/\347\263\273\347\273\237\346\265\213\350\257\225\347\224\250\344\276\213.md"
create mode 100644 src/test/java/cn/edu/whut/sept/zuul/integration/T5SystemPathTest.java
```

图 4-8: `git commit` 命令及提交信息

6. 推送至远程分支

执行 `git push origin feature/lj`，将本地提交推送到远程 `feature/lj` 分支。

```
D:\softwaretest\kai-fa-519club>git push origin feature/lj
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (14/14), 6.73 KiB | 1.68 MiB/s, done.
Total 14 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/wutkst/kai-fa-519club.git
c7a3674..c098f97 feature/lj -> feature/lj
```

图 4-9: git push 执行结果

7. 发起 Pull Request 并等待 CI 与 Review

在 GitHub 上从 feature/lj 向 dev 发起 Pull Request，CI 门禁（mvn checkstyle:check → mvn test → mvn package）自动触发，全部通过后等待小组成员 Review，通过后合并进 dev，对应 Issue 自动关闭。

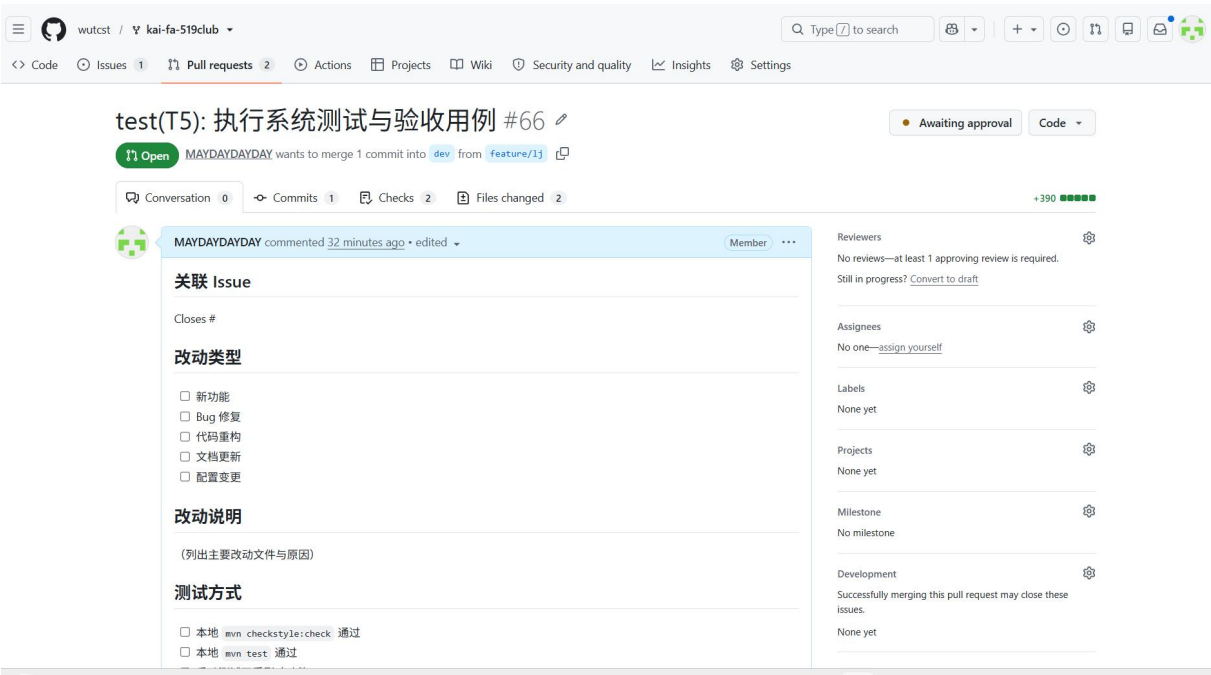


图 4-10: GitHub PR 页面及 CI 检查通过状态

4.2.3 任务实施过程

（一）E1 use 命令（Issue #18）

任务说明： 实现 use <物品> 命令，根据当前房间和使用的物品产生不同效果，包括在超市用“湿漉漉的三十元钱”兑换一卡通、在博学西楼持锤子砸开生锈门锁，以及对秒表、祈福黄纸等干扰物使用时触发彩蛋提示。use 命令是后续 combine、submit 等

命令的前置依赖。

实施过程

实施思路：use 命令的核心是“当前在哪个房间”和“用的是什么物品”两个维度的组合。采用命令模式新建 UseCommand 类实现 CommandInterface 接口，执行时先从玩家背包中查找物品，再按房间 ID 路由到对应处理方法，最后成功操作后通过 ActionTimeCost.deduct() 扣除耗时。干扰物品单独封装为 tryUseGlobalDistraction() 方法，在进入房间路由前优先处理，保持主流程清晰。

实验操作：

UseCommand.execute(game, itemName):

item = player.findItemInInventory(itemName)

if item == null → 提示"你没有该物品"，返回

if tryUseGlobalDistraction(game, item) → 扣时，返回 // 干扰物彩蛋

if item == "一块遗弃的秒表" → 打印剩余时间，扣时，返回

roomId = player.getCurrentRoom().getRoomId()

switch roomId:

case "supermarket" → useInSupermarket(player, item) // 三十元换一卡通

case "boxue_west" → useInWestBuilding(game, player, item) // 锤子砸锁

default → 提示"在这里无法使用"

useInSupermarket(player, item):

if item 不是"湿漉漉的三十元钱" → 提示只能使用三十元钱

if player 已有一卡通 → 提示已有

else → 移除三十元，addItem(一卡通)

useInWestBuilding(game, player, item):

if item 不是"锤子" → 提示只能用锤子

if currentLevel < 3 → 提示不需要砸锁

if 门锁已解除 → 提示不需要再用锤子

else → 移除锤子，levelManager.unlockWestBuildingExit()

实验中遇到的问题及解决方法：

问题一： 玩家输入 use 三十元钱（省略“湿漉漉的”前缀）时系统提示“你没有该物品”，但背包里明明有，导致第一关无法换卡。排查发现物品名是完整的“湿漉漉的三十元钱”，字符串严格匹配失败。

解决： 在 isMoneyItem() 方法中同时匹配完整名称和简写“三十元钱”，均不区分大小写，两种输入都能正确触发兑换。

问题二： 第二关中，西楼门锁尚未启用，但玩家持锤进入西楼执行 use 锤子 时错误地触发了砸锁逻辑。

解决： 在 useInWestBuilding() 中增加关卡判断，getCurrentLevel() < 3 时直接提示“这里的门锁暂时不需要砸开”，第三关前彻底屏蔽该逻辑。

实验提交步骤：

1.git status

实验结果： 本地运行 mvn checkstyle:check、mvn test、mvn package 三项门禁全部通过。第一关超市可正常兑换一卡通，第三关及以上博学西楼持锤子可砸开门锁，干扰物品各有对应彩蛋文本，命令逻辑分支覆盖完整。

（二）E8 NPC 对话（Issue #25）

任务说明： 实现 talk 命令，玩家可与当前房间的 NPC 对话，不同房间、不同关卡下 NPC 给出差异化提示或发放物品。教育超市宿管阿姨告知换卡流程；博学北楼志愿者在第二、三关直接发放归寝单，第四关起引导前往图书馆；图书馆工作人员从第四关起可发放归寝单。

实施过程

实施思路： NPC 行为同时依赖房间和关卡两个维度，如果把所有分支全写在命令类里会很乱。因此将所有 NPC 对话逻辑集中到 NpcService 工具类中，以 roomId 作

为 switch 的主键，各房间处理函数内再按关卡号细分，发放归寝单的动作封装为 issueDormForm() 复用。

实验操作：

NpcService.talk(game):

```
roomId = player.getCurrentRoom().getRoomId()
```

```
level = levelManager.getCurrentLevel()
```

switch roomId:

```
case "supermarket" → 打印宿管提示"交三十元可兑换一卡通"
```

```
case "boxue_north" → talkVolunteer(game, level)
```

```
case "library" → talkLibrarian(game, level)
```

```
default → "这里没有可以对话的人"
```

talkVolunteer(game, level):

```
level == 1 → 提示本关无需归寝单
```

```
level 2 ~ 3 → issueDormForm(player, "志愿者...发放了归寝单")
```

```
level >= 4 → 提示"请前往图书馆领取"
```

issueDormForm(player, msg):

```
if player 已有归寝单 → 提示已领过
```

```
else → player.takeItem(new Item("归寝单", 5)); 打印成功信息
```

实验中遇到的问题及解决方法：

问题： 调试时发现第四关玩家对博学北楼志愿者执行 talk 后仍然能领到归寝单，实际上第四关应该只能从图书馆领取。

解决： 检查代码后发现 level >= 2 的范围把第四关也包含进去了。将条件收紧为 level >= 2 && level <= 3 才走志愿者发放逻辑，level >= 4 直接跳到引导提示，不再发放物品，测试后行为符合策划。

实验结果： 三项本地门禁测试通过。各关卡各房间 NPC 行为与策划一致：第一关志愿者仅提示信息，第二、三关正常发放归寝单，第四、五关引导至图书馆，宿管阿姨

始终给出换卡提示，归寝单不重复发放。

（三）E3 combine 命令（Issue #20）

任务说明： 实现 combine 命令，玩家在博学西楼内，背包同时持有棍子、石头、绳子三样材料，即可消耗三件材料合成锤子；锤子是用 use 命令砸锁出门的必要道具，形成物品链路。

实施过程

实施思路： combine 的前置条件只有两个：在正确房间、材料齐全。逻辑相对独立，直接在 CombineCommand 内完成，不需要额外服务类。执行顺序是先检查房间、再检查是否已有锤子（避免重复合成）、再检查三件材料，通过后移除材料并创建锤子物品。

实验操作：

CombineCommand.execute(game, _):

```
if room.getRoomId() != "boxue_west" → "只能在博学西楼内合成锤子"
```

```
if player.hasItem("锤子") → "你已经有一把锤子了"
```

```
if !hasAllMaterials(player) → "材料不齐：需要棍子、石头和绳子"
```

```
remove(player, "棍子")
```

```
remove(player, "石头")
```

```
remove(player, "绳子")
```

```
player.takeItem(new Item("锤子", 200))
```

```
ActionTimeCost.deduct(COMBINE)
```

hasAllMaterials(player):

```
return hasItem("棍子") && hasItem("石头") && hasItem("绳子")
```

实验中遇到的问题及解决方法：

问题： 合成完成后，联调 use 砸锁时，发现锤子被 useInWestBuilding() 移除后，玩家重新进入西楼仍可再次合成（材料已不存在，理论上应提示材料不足），但测

试中出现合成成功提示却未生成锤子的奇怪现象。

解决： 排查发现是测试构造数据时误将材料重复 add 到背包，造成数量异常，代码本身逻辑正确。同时确认西楼困锁在 unlockWestBuildingExit() 调用后永久解除，本身不会出现多次砸锁的情况。补充了对应边界测试用例，验证砸锁后 isWestBuildingExitLocked() 返回 false。

实验结果： 三项本地门禁测试通过。三件材料齐全时合成成功，原材料从背包移除，锤子正常添加；材料不齐或不在西楼时均有明确提示；锤子可被 use 命令正常砸锁，物品链路完整。

（四）E4 unlock 命令（Issue #21）

任务说明： 实现 unlock<密码> 命令，对应两处密码解锁场景：体育馆器材室（密码 WHUT2026，第三关起生效，解锁后产出手电筒）和第五关寝室智能锁（密码 20000527，输错扣 60 秒）。食堂占座纸条上的日期 2026.06.01 为干扰密码，输入时给出专属提示“别被纸条骗了”。

实施过程

实施思路： UnlockCommand 只做命令接收，将实际逻辑全部委托给 UnlockService，职责分离。UnlockService 按房间 ID 分两条路径：体育馆路径密码正确后动态向房间添加手电筒物品；寝室路径密码错误有固定 60 秒惩罚并记录错误次数；干扰密码提前单独匹配，区别于普通错误密码给出不同文本。

实验操作：

UnlockService.unlock(game, password):

```
roomId = player.getCurrentRoom().getRoomId()
if "gymnasium" → unlockGym(game, password)
if "dormitory" → unlockDormitory(game, password)
else           → "这里无法输入密码"
```

unlockGym(game, password):

```

if level < 3 → "本关无需解锁，手电可直接拾取"
if already unlocked → "器材室已解锁"
if password == "WHUT2026" →
    markGymStorageUnlocked()
    if !gym.containsItem("手电筒") → gym.addItem(手电筒)    // 避免重复添加
else → handleWrongPassword(false)
unlockDormitory(game, password):
    if level != 5 → "本关无需解锁"
    if already unlocked → "已打开"
    if password == "20000527" → markDormitoryPasswordUnlocked()
    else → handleWrongPassword(true)    // 扣 60 秒
handleWrongPassword(dormitoryLock):
    if password in {"2026.06.01", "20260601"} → "别被纸条骗了"
    else → "密码错误"
    deduct(60s if dormitoryLock else WRONG_PASSWORD)

```

实验中遇到的问题及解决方法：

问题： 体育馆解锁成功后，手电筒通过 `gym.addItem()` 放入房间，但若玩家已提前从体育馆捡走了场景初始手电筒，再次密码解锁时会重复添加一把，背包最终出现两把手电筒。

解决： 添加前先用 `gym.containsItem(FLASHLIGHT_ITEM)` 检查房间内是否已有手电筒，有则跳过 `addItem`，确保无论场景初始状态如何，房间内手电筒数量始终最多一把。

实验结果： 三项本地门禁测试通过。体育馆密码正确后可拾取手电筒，第五关寝室密码正确可开锁，干扰密码输入有专属提示，密码错误按规则扣时，相关测试用例全部断言通过。

（五）E2 submit 命令（Issue #19）

任务说明： 实现 submit <物品> 命令，玩家在教育超市（寝室门口）完成归寝手续。正确提交归寝单或退寝条后，寝室条件门解除；若提交错误物品（如“别人饭卡”）则本关直接失败。此命令依赖 E1 use 命令和 E13 条件门的关卡配置。

实施过程

实施思路： submit 进入核心逻辑前有三道前置检查：必须在超市房间内、当前关卡要求提交归寝单（requiresDormitorySubmit()）、且尚未提交过。这样处理的好处是在第一关等无需提交的关卡中，玩家误触 submit 不会造成任何影响。错误物品触发的失败直接调用 levelManager.failCurrentLevel()，与超时失败走同一路径，确保失败逻辑统一。

实验操作：

SubmitCommand.execute(game, itemName):

```
if room.getRoomId() != "supermarket" → "请前往教育超市寝室门口"
```

```
if !levelConfig.requiresDormitorySubmit() → "本关无需提交归寝单"
```

```
if isDormitorySubmitCompleted() → "你已经提交过归寝单了"
```

```
item = player.findItemInInventory(itemName)
```

```
if item == null → "你没有该物品"
```

```
if isValidSubmitItem(itemName):      // "归寝单" 或 "退寝条"
```

```
    player.removeItemFromInventory(item)
```

```
    levelManager.markDormitorySubmitCompleted()
```

```
    ActionTimeCost.deduct(SUBMIT)
```

```
else:
```

```
    println("提交错误！本关失败")
```

```
    levelManager.failCurrentLevel()
```

实验中遇到的问题及解决方法：

问题： 第一关测试时，玩家在超市执行 submit 归寝单 后提示“本关无需提交归寝单”，但归寝单仍留在背包里，不清楚是不是逻辑问题。

解决： 对照策划文档确认第一关确实不需要 submit，属于正常流程，并非 Bug。实际是测试时对关卡策划理解有误。厘清后在单元测试中专门补充了“第一关 submit 应被跳过”的断言，防止以后误改逻辑。

实验结果： 三项本地门禁测试通过。第二关及以上需要 submit 的关卡中，正确提交归寝单后门禁解除可正常归寝；提交错误物品时本关失败逻辑触发；无需提交的关卡操作 submit 有清晰提示，不影响游戏状态。

（六）F7 Vue 图形界面（Issue #30）

任务说明： 基于 Vue 3 + TypeScript 搭建游戏图形前端 vue-portal，实现单机模式（/solo）与联机模式（/multiplayer）的完整场景 UI，包含房间场景图与切换动画、方向导航按钮、背包 HUD、倒计时、NPC 对话弹窗、物品检视弹窗、过关/失败结果覆盖层等。支持经典与沉浸两种显示模式切换。

实施过程

实施思路： 整体采用 Vue 3 Composition API，按照职责将 UI 拆分为独立组件：GameScene.vue 负责场景图渲染与切换动画；DirectionNav.vue 处理方向按钮；InventoryHudSolo.vue/InventoryHud.vue 管理背包格显示；HudTimer.vue 负责倒计时。SoloGameView.vue 作为单机模式顶层 View，组合所有子组件，通过 useGameController.ts 与后端 Spring Boot REST API 通信。路由层面，Vue Router 管理 /solo、/multiplayer、/home 等页面，组件间以 defineProps / defineEmits 通信，跨层状态通过 Pinia 管理。

实验操作：

房间切换动画根据显示模式差异化处理：经典模式统一使用淡入淡出，沉浸模式依据移动方向（north/south/east/west）使用对应方向滑入动画，back 命令使用反向动画 room-back。

// GameScene.vue 动画选择

```
transitionName = computed(() => {
```

```
if (!isImmersive || !moveDirection) return 'room-fade'
if (moveDirection === 'back')      return 'room-back'
return `room-slide-${moveDirection}`
}))
```

roomLayoutDefaults.ts 中为每个房间维护物品和 NPC 的相对坐标映射，前端可在不额外调用后端接口的情况下渲染物品位置。背包显示移除了原有的固定槽位上限，改为根据玩家实际背包列表动态渲染，避免物品过多时被截断。

实验中遇到的问题及解决方法：

问题一： 沉浸模式下南侧方向导航箭头与房间描述文字出现视觉重叠，在屏幕较小时尤为明显，影响操作体验。

解决： 在 DirectionNav.vue 中为南侧按钮引入 --south-offset-x CSS 变量，默认水平偏移 44px，将箭头位置与文字错开，小屏也不再重叠。

问题二： 联机前端初期接入时，命令发送后后端已返回成功，但前端场景仍停留在旧房间，未刷新。

解决： 排查 useGameRoomController.ts 发现命令发送后缺少对状态刷新接口的调用，补充 await fetchRoomState() 后每次命令返回时立即拉取最新房间和背包状态，页面同步正常。

实验结果： 三项本地门禁测试通过。以下为部分实验截图：

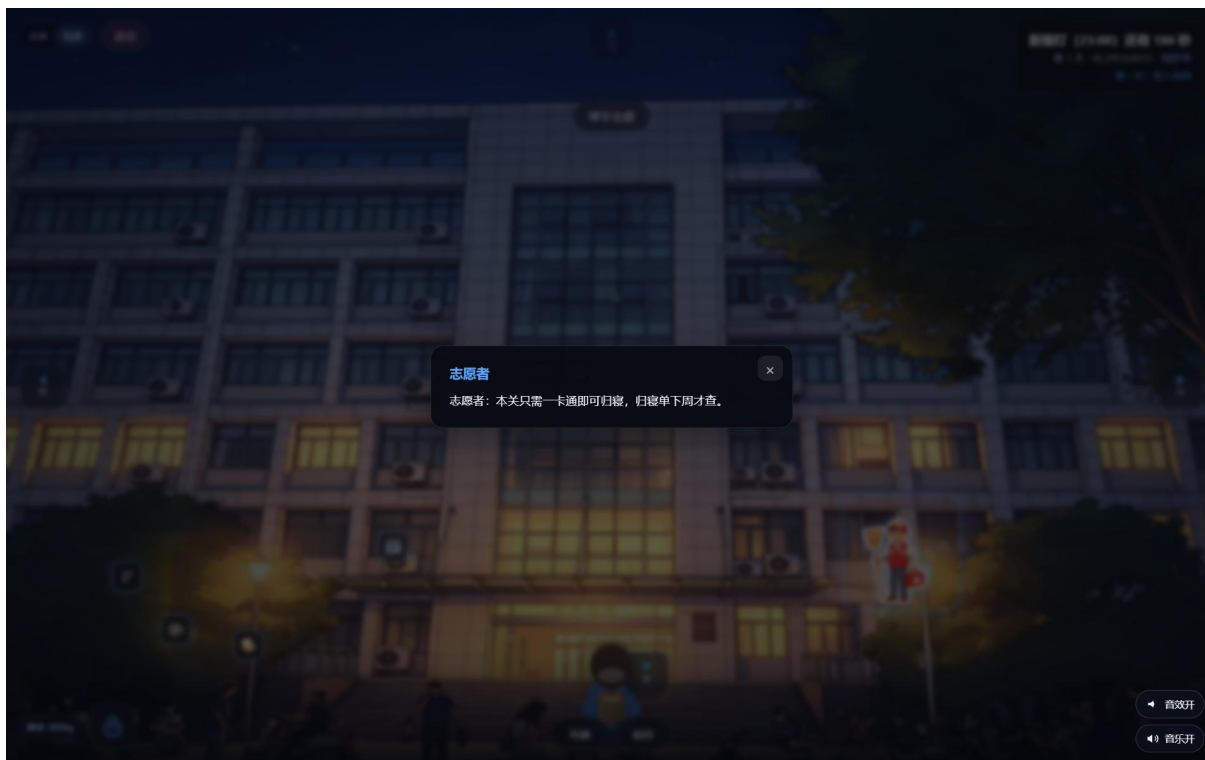


图 4-10：单机模式沉浸视图——沉浸场景动画效果与 NPC 对话弹窗

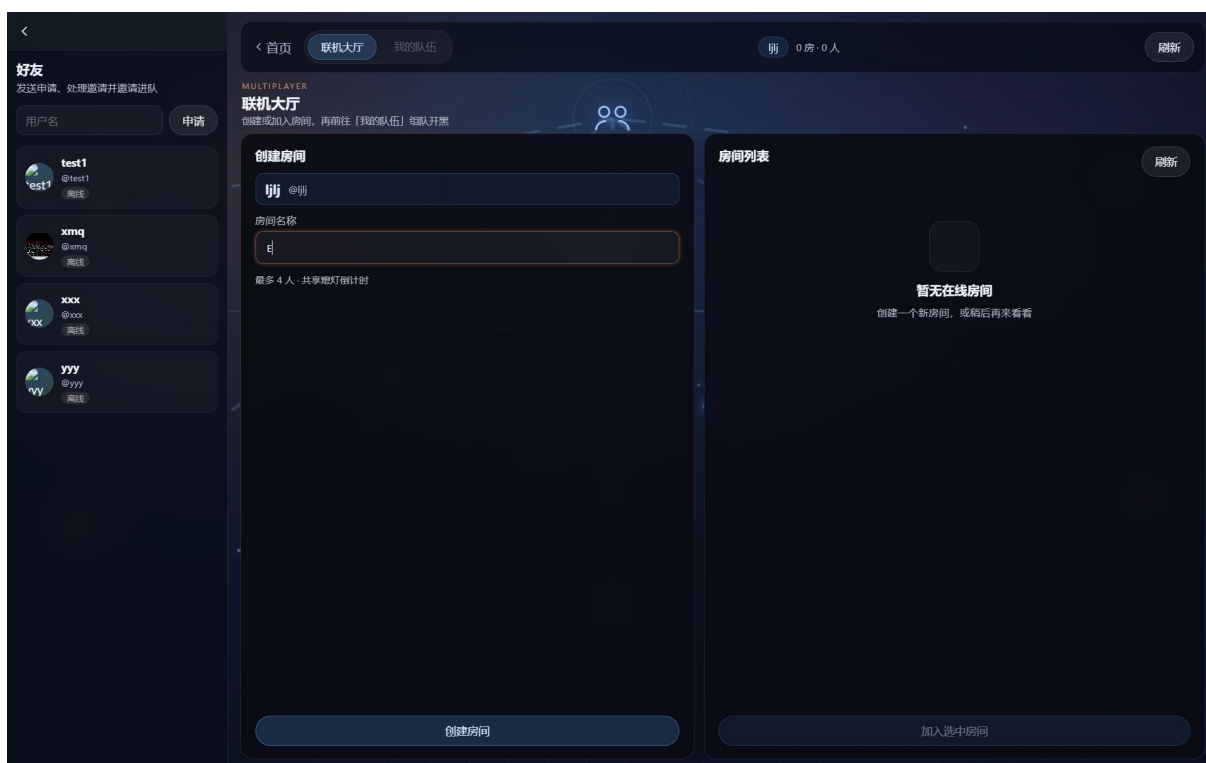


图 4-11：联机大厅

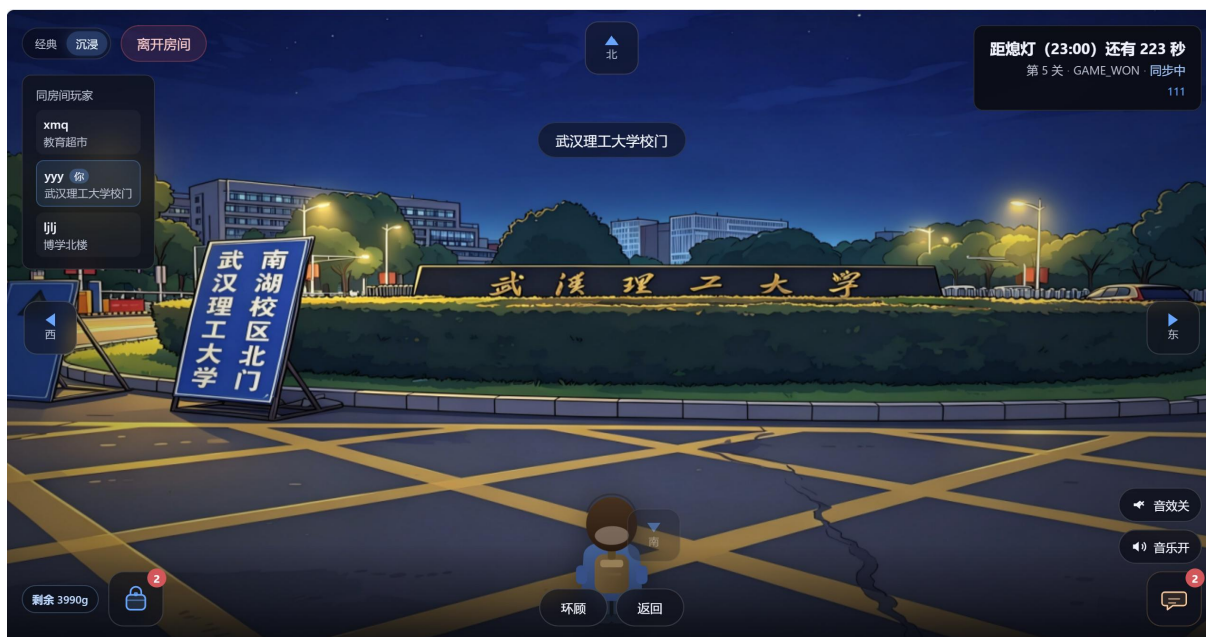


图 4-12：联机界面

基于 Vue 3 Composition API + TypeScript 构建了完整游戏 Web 前端，涵盖单机与联机双模式、经典与沉浸双视图，组件结构清晰，前端交互逻辑与后端游戏内核规则保持一致。

（七）F8 H2 存档——DAO 层协作（Issue #31）

任务说明：与肖梦琪协作，负责 H2 持久化层的 DAO 实现部分，包括背包物品明细表（SaveItemRepository）与通关记录表（ClearRecordRepository）的 SQL 读写，以及 Vue 前端存档触发与读档恢复逻辑。

实施过程

实施思路：存档数据拆分为三张表：主存档表（关卡号、剩余秒数、当前房间）、物品明细表（背包中每件物品的描述和重量）、通关记录表（用户、关卡、通关时间）。各 Repository 类只负责单表 CRUD，上层 GamePersistenceService 负责将三张表的操作组合为一次完整的存/读档事务，前端通过 /api/solo/save 和 /api/solo/load 接口触发。

实验操作（协作部分）：

SaveItemRepository.saveItems(saveId, items):

for item in items:

```
INSERT INTO save_items(save_id, description, weight) VALUES (?, ?, ?)
```

SaveItemRepository.loadItems(saveId):

```
SELECT description, weight FROM save_items WHERE save_id = ?
```

→ 返回 List<Item>

ClearRecordRepository.insertRecord(userId, level, clearTime):

```
INSERT INTO clear_records(user_id, level_num, clear_at) VALUES (?, ?, ?)
```

Vue 前端读档后将返回的关卡号、剩余秒数和背包物品列表写入 Pinia store, UI 自动响应式刷新, 恢复存档时的游戏现场。

实验结果: 三项本地门禁测试通过。存档后关闭游戏, 重新读档可恢复关卡号、剩余秒数和背包物品; 第五关通关后通关记录正常写入; 存档单元测试以 H2 内存库运行, CI 环境隔离无污染。

(八) F6 联机模式——Vue 前端协作 (Issue #29)

任务说明: 与肖梦琪协作, 负责联机模式的 Vue 前端 (/multiplayer 路由), 包括多人房间场景、其他玩家状态显示、命令发送与状态同步、联机倒计时 (以服务端时间为准)。

实施过程

实施思路: 联机前端复用了单机的 GameScene.vue、DirectionNav.vue、InventoryHud.vue 等组件, 差异在于需要实时拉取其他玩家的位置与背包状态。采用轮询方式每秒调用 /api/game/state 接口, 无需引入 WebSocket, 实现相对简单。倒计时显示以服务端返回的 remainingSeconds 为准, 前端不自行计算, 防止多端时间漂移。

实验操作:

```
// useGameRoomController.ts 命令发送流程
```

```
async sendCommand(cmd):
```

```
await POST /api/game/command { roomId, playerId, command: cmd }  
await fetchRoomState() // 拉取最新状态刷新 UI  
// 轮询状态同步  
setInterval(fetchRoomState, 1000)
```

实验结果： 三项本地门禁测试通过。

运用 Vue 3 与 Spring Boot REST API 对接实现了至少两人联机的基础功能，命令发送后房间状态多端同步刷新，计时以服务端为准，联机模式对单机流程无影响。

（九）T2 拓展命令单元测试（Issue #33）

任务说明： 为本人负责的五项拓展命令（use、submit、combine、unlock、NPC talk）各编写 JUnit 5 单元测试，每个命令至少覆盖正向用例和异常用例。

实施过程

实施思路： 测试类参照项目已有的 TestUtils 工具类，构造简化的 Game 实例（含 Player、LevelManager 等必要组件）隔离控制台 I/O，不依赖 GUI 或网络。各 Command 的测试类单独建立，测试方法命名清晰区分正向和异常场景，便于 Surefire 报告定位。

关键测试用例：

UseCommandTest:

```
testUseMoney_InSupermarket_GivesCampusCard() // 超市用三十元→获得一卡通  
testUseMoney_NotInSupermarket_Fails() // 错误房间→失败
```

CombineCommandTest:

```
testCombine_AllMaterials_CreatesHammer() // 材料齐全→合成成功  
testCombine_MissingMaterial_Fails() // 材料不齐→失败
```

UnlockCommandTest:

```
testUnlockGym_CorrectPassword_AddsFlashlight() // 正确密码→产出手电筒  
testUnlockDormitory_WrongPassword_DeductsTime() // 错误密码→扣 60 秒
```

```
testUnlockDormitory_DistractioPassword_SpecialHint() // 干扰密码→专属提示
SubmitCommandTest:
    testSubmit_ValidItem_MarksCompleted()           // 正确提交→归寝标记完成
    testSubmit_WrongItem_FailsLevel()               // 错误物品→关卡失败
```

实验结果： `mvn test` 全部通过，各命令正向与异常分支均有断言覆盖，CI 门禁绿灯。新增测试类随功能 PR 一并提交，Surefire 报告无失败用例。

（十）T3 GUI 与存档单元测试（Issue #34）

任务说明：对 F7 Vue GUI 的后端桥接层（GameGuiController、GuiPhase3Helper、GuiOutcomeHelper 等）及 F8 H2 存档 DAO 编写单元测试，保证在 CI headless 环境下可正常运行，不依赖真实 GUI 渲染。

实施过程：

GUI 桥接层测试重点验证 API 返回数据格式正确（房间信息字段、命令响应状态等），直接调用 Controller 方法断言返回值，无需启动完整 Spring 容器。存档测试使用 H2 内存库（`jdbc:h2:mem:test`），每个测试方法前重置数据库，保证用例间互不干扰。存档链路测试验证“保存→模拟重启→读档”的完整流程，断言恢复后的关卡号、剩余秒数、背包物品与存档前一致。

实验结果： `mvn test` 全部通过，GUI 桥接测试和存档测试均在 headless CI 环境下正常运行，存档读写数据一致性验证通过。

（十一）T6 联机回归测试——协作（Issue #37）

任务说明：与肖梦琪协作，在联机模式代码合并进 dev 后，验证单机五关主流程不受联机改动影响，同时记录联机演示场景（至少两名玩家同时在线、房间状态同步、服务端计时）。

实施过程：

联机 PR 合并后，本人在本地分别以单机和联机两种模式跑通第一关，记录关键操作节点（换卡、提交归寝单、归寝）的状态截图，确认 dev 分支 CI 连续绿灯。测试过程中发现多人模式下关卡重开时背包未全部清空的问题，定位到 `Game.clearAllInventories()` 未对所有联机玩家执行，修复后补充了 `MultiplayerGameTest` 中的回归用例，CI 恢复正常。

实验结果： 三项本地门禁测试通过。单机与联机各跑通第一关验收，联机背包清空 Bug 修复后回归测试通过，dev 分支 CI 持续绿灯。

4.2.4 个人实验成果总结

本人在本次项目中共完成拓展命令 E1—E4、E8 五项后端功能，主导 F7 Vue 图形界面开发，协作参与 F6 联机前端与 F8 H2 存档，并主责 T2、T3 两项单元测试。最终交付的代码均已通过本地 `mvn checkstyle:check`、`mvn test`、`mvn package` 三项门禁，相关 PR 经评审合并进 dev 分支，对应 GitHub Issue 全部关闭。Vue 前端实现了单机与联机双模式、经典与沉浸双视图，游戏五关主流程可完整运行，拓展命令单元测试覆盖正向与异常分支，CI 持续保持绿灯状态。

表 4-4 个人实验成果总结表

任务编号	交付内容	验收状态
E1—E4、E8	UseCommand、SubmitCommand、CombineCommand、UnlockCommand、NpcService 及对应测试类	mvn test 通过，PR 已合并，Issue 已关闭
F7	vue-portal 单机/联机双模式、经典/沉浸双视图	PR 已合并，Issue 已关闭
F6（协作）	联机模式 Vue 前端（/multiplayer 路由）	PR 已合并
F8（协作）	H2 存档 DAO 层（SaveItemRepository、ClearRecordRepository）	PR 已合并
T2	拓展命令单元测试（UseCommandTest 等 5 个测试类）	CI 通过
T3	GUI 桥接与存档单元测试	CI 通过

4.3. 成员三任务分工及实施过程描述

成员姓名： 彭慧星

主要角色与责任： 关卡内容与多人服务端开发。根据小组分工，主要负责第四关的详细设计与实现、部分核心底层机制（黑暗罚时、困锁）的联调、第五关传送功能的启用，以及网络多人模式服务端的开发与协议制定。

任务一：第四关游戏内容设计与实现

任务描述： 基于组长肖梦琪提供的 LevelManager 和 Room 规则底层，完成第四关（图书馆主题）的具体游戏内容配置。此关的核心难点在于引入“博学主楼黑暗区域”的惩罚机制，以及“图书馆归寝单”的准入与提交逻辑。

实施过程：

需求分析与配置： 仔细研读会议纪要中关于第四关的策划案，明确了本关需要实现的特定物品（归寝单、手电筒）、NPC（图书馆志愿者）以及特殊规则（黑暗罚时）。

黑暗罚时机制联调（E12）： 与组长肖梦琪紧密合作，在其开发的 Room 扩展规则基础上，完成了“黑暗区域”的判断逻辑。实现了当玩家未携带“手电筒”而进入“博学主楼”房间时，系统自动扣除 60 秒游戏时间，并强制将玩家传送回上一安全区域的功能。通过单元测试覆盖了多种边界情况（如携带手电筒进入、罚时后时间归零导致游戏失败等）。

条件门与 NPC 交互（E13, E8）： 配置了第四关图书馆入口的条件门，确保玩家必须持有“一卡通”才能进入。同时，实现了与“图书馆志愿者”NPC 的对话逻辑（talk 命令），玩家需通过正确的对话选项才能领取到本关通关必需的“归寝单”。

关卡数据配置： 在 LevelManager 的配置文件中，为第四关设定了正确的初始房间、可用出口集合、干扰物（如“过期的杂志”、“占座的笔记本”）位置以及通关条件（必须持有“归寝单”并在寝室执行 submit 命令）。

任务二：网络多人模式（F6）服务端开发

任务描述： 设计并实现一个轻量级的 Socket 服务端，支持多个客户端实例连接，实现游戏世界状态的同步（如玩家位置、房间物品变化）以及熄灯倒计时的统一广播，以满足课程对网络多人功能的要求。

实施过程：

协议设计： 首先与组长肖梦琪确定客户端与服务端的通信协议。采用了基于 JSON

的文本消息格式，定义了 LOGIN、COMMAND、STATE_UPDATE、TIME_SYNC 等核心消息类型，确保了协议的可扩展性和可读性，并在设计文档中详细描述了消息结构，对标了 RESTful 风格的设计思想。

服务端核心实现：使用 Java 的 ServerSocket 和线程池，构建了一个能够并发处理多个客户端请求的 TCP 服务端。核心功能包括：客户端连接与登录管理：维护一个在线客户端列表，处理昵称注册和会话管理。命令转发与状态同步：接收任一客户端发来的游戏命令（如 go east, take book），在服务端维护的权威游戏世界实例上执行该命令，然后将执行后的最新状态（如玩家新位置、房间物品变化）广播给所有已连接的客户端。计时器同步（E10）： 确保熄灯倒计时由服务端统一维护，并定期（例如每秒）向所有客户端广播剩余的秒数，保证多玩家看到的倒计时一致。

集成与测试： 将服务端与主游戏项目进行集成，并编写了简单的多客户端测试脚本，模拟多个玩家同时登录、移动、拾取物品的场景，验证了服务端状态同步的正确性和并发处理的稳定性。

任务三：第五关传送干扰（E17）与西楼困锁联调（E14）

任务描述： 启用并配置第五关中的“体育馆传送干扰”功能，该功能复用已有的 TeleportRoom 类。同时，协助刘晶完成第三关“西楼困锁”机制中，关于条件门与房间状态变化的联调工作。

实施过程：

体育馆传送配置： 在第五关的地图配置中，将“体育馆”房间设置为 TeleportRoom 类型，并将其可能传送的目标房间列表配置为第五关地图中的其他几个关键房间（如“操场”、“教学楼”），以此增加终章的探索难度和随机性。通过模拟测试确保传送功能按预期触发，且传送后玩家的当前房间被正确更新。

西楼困锁机制联调： 与刘晶合作，在其实现的 combine 和 use 命令基础上，验证了“被困西楼”场景的完整性。即：玩家进入“西楼”后，出口被锁定；只有当玩家使用合成的“锤子”use 在“西楼大门”上时，出口才会被正确解锁。测试了包括“未持锤子退出”、“使用错误物品”等多种异常流程，确保逻辑严谨。

通过以上工作，彭慧星成功交付了第四关的完整玩法与数据，为游戏提供了丰富的关卡体验；开发并运行了网络多人服务端，使项目具备了基础的联机协作能力；并协作

完成了西楼和体育馆等关键特殊房间机制的最终集成与测试,有力支撑了项目在 6 月 20 日前五关贯通和 F6 功能可演示的总体目标。

4.4. 成员四任务分工及实施过程描述

成员姓名：庞绮君

主要角色与责任：负责项目中若干核心规则的落地与终章内容配置，并承担测试验收与文档整理工作。具体包括：E15 按关卡解锁房间出口、E14 西楼困锁与 E7 魔法饼干加时、E16 第 1—2 关及第 5 关场景数据，以及 T5 系统测试与 T7 测试报告撰写（开发分支 feature/pqj）。

任务一：按关卡开放地图（E15）

任务描述：

游戏采用固定十房地图，但各关可探索范围不同。本任务要求在 LevelManager 提供的关卡号基础上，实现从第一关五个基础场景到第五关全图开放的渐进机制，并与后续困锁、传送等规则衔接。

实施过程：

配置层设计：在 LevelConfig 中为每关维护开放房间集合 UNLOCKED_ROOM_IDS，通过 isRoomUnlocked() 判断目标房间是否已解锁；未开放时统一使用提示文案“夜色中，这个方向暂未开放”。

移动拦截：在 Game.isRoomAccessible() 中接入上述判断；GoCommand 在玩家试图前往未开放房间时输出锁定提示并阻止移动。与图形界面同学联调，保证文本模式与 Vue 界面中未开放方向的处理一致。

测试验证：编写 LevelConfigRoomUnlockTest、RoomUnlockTest 覆盖五关各房间开放状态；在 GameIntegrationTest 中验证第一关体育馆、食堂不可进入，第二关起可正常通行。

任务二：西楼困锁与魔法饼干加时（E14、E7）

任务描述：

第三关引入西楼密室解谜：玩家进入西楼后东侧出口自动锁定，须收集材料合成锤

子并砸锁方可离开；同时完善魔法饼干规则——每关首次食用增加 300 秒并提升负重，同关重复食用不再加时。

实施过程：

西楼困锁（E14）：在 LevelManager 中维护困锁与破门两个状态位。玩家进入西楼时触 onEnterWestBuilding()，第三关起自动困锁；GoCommand 通过 Game.isTrappedInWestBuilding() 拦截向东移动。与刘晶联调 CombineCommand 与 UseCommand，砸锁成功后调用 unlockWestBuildingExit() 解除困锁。WestBuildingLockTest 覆盖第三关以下不触发、无锤无法离开、砸锁后可正常出门等边界场景。

魔法饼干加时（E7）：在 LevelManager 中增加 magicCookieBonusUsed 标志；EatCookieCommand 首次食用时调用 LevelTimer.addSeconds(300) 并标记已使用，重复食用仅增负重。与彭慧星联调 FeedCommand 喂猫得饼干路径，EatCookieCommandTest 验证首次加时、重复不加时等规则。

任务三：关卡内容配置与系统测试（E16、T5、T7）

任务描述：

负责第一、二关及第五关（终章）的房间公告、干扰物品与解谜线索配置；完成系统测试，编写用例、执行五关主路径与失败路径验收、写测试报告。

实施过程：

关卡数据配置（E16）：在 LevelRoomContent 中实现按关累积 patch 策略——公告与干扰物从第 1 关回放至当前关，合成完整地图状态。第一、二关配置北楼志愿者线索、社团传单、赛事纪念帽等新手向干扰物；第五关配置图书馆校史公告（引导推理密码 20000527）、寝室智能锁提示、体育馆传送说明，以及金块、外卖等终章干扰物。

系统测试（T5）：与彭慧星协作编写系统测试用例.md，并实现 T5SystemPathTest 自动化覆盖五关主路径。2026 年 6 月 8 日前完成首轮执行：mvn test 340 项全通过；人工完成 Vue 经典版与沉浸版第一关至第五关全通及读档、西楼困锁提示等 GUI 场景试玩。

测试报告（T7）：汇总 T1—T6 的测试结果，撰写实训测试报告相关章节，数据与 Surefire 报告及系统测试用例表保持一致。

通过以上工作，庞绮君在配置层与命令层交付了按关卡开放地图的基础机制；在 LevelManager 与 EatCookieCommand 中实现了西楼困锁与魔法饼干加时并完成跨成员联调；在 LevelRoomContent 中配置了第 1—2 关及第 5 关终章内容；完成了 T5 系统测试并整理 T7 测试报告。

5. 实施过程问题记录与分析

本节对项目开发过程中遇到的主要问题进行记录，涵盖个人功能开发中的逻辑问题、队友模块中发现的缺陷，以及多人协作、前后端联调过程中出现的集成问题。

问题一：use 命令物品名称精确匹配导致换卡失败

发现过程：手动测试第一关时，输入 use 三十元钱 系统提示“你没有该物品”，背包里明明有“湿漉漉的三十元钱”，但字符串精确比对失败，第一关主线流程因此卡死。

原因分析：最初 UseCommand 中直接用 `item.getDescription().equals(itemName)` 做匹配，玩家输入的简写与存储的完整名称不一致时就无法识别。

解决方案：将名称校验提取为 `isMoneyItem()` 方法，同时接受“湿漉漉的三十元钱”和“三十元钱”两种形式，并改为 `equalsIgnoreCase` 不区分大小写。此后统一对所有命令的物品名比较方式做了排查，确保其他命令不存在同类问题。

问题二：NPC 对话志愿者可在第四关仍发放归寝单

发现过程：T2 单元测试中发现 `NpcCommandTest` 在第四关对志愿者 talk 后仍然获得了归寝单，而策划要求第四关起归寝单只从图书馆领取。

原因分析：`talkVolunteer()` 中判断志愿者发放归寝单的条件写的是 `level >= 2`，没有设置上界，导致第四、五关也会走进该分支。

解决方案：修改为 `level >= 2 && level <= 3`，第四关及以上直接跳到“请前往图书馆”的提示分支，不再触发发放逻辑。同时在测试用例中覆盖了关卡 2、3、4、5 四个边界情况的 NPC 行为断言。

问题三：Vue 联机前端命令发送后场景未刷新

发现过程：联机模式初期联调时，玩家输入 go north 后命令已发给后端并返回成功，但前端页面房间名和场景图没有变化，仍停留在旧房间。

原因分析：`useGameRoomController.ts` 里命令发送的 POST 请求结束后，没有接着拉取最新的游戏状态，前端 Pinia store 里的房间 ID 没有得到更新，自然也就没有触发组件的响应式刷新。

解决方案：在命令发送函数中，POST 完成后立即 `await fetchRoomState()`，用服务端返回的最新状态刷新本地 `store`。调整后再测试，移动、`take`、`use` 等各类命令后场景均可实时同步。

问题四：体育馆传送在第四关被意外触发（彭慧星，E17）

发现过程：T2 测试阶段手动跑第四关主流程，发现进入体育馆后触发了随机传送提示，而策划中体育馆传送只在第五关开启。`GymnasiumTeleportTest` 的 `testGymnasiumTeleportDisabledBeforeLevelFive` 用例也因此不通过。

原因分析：`TeleportRoom` 的 `isTeleportEnabled()` 判断最初绑定的是“已解锁出口数量”而非严格的关卡号，第四关解锁了体育馆方向后意外满足了启用条件。

解决方案：将判断改为直接读取 `levelManager.getCurrentLevel() == 5`，确保传送机制与关卡号绑定，而不依赖地图出口的解锁状态。修复后第四关进体育馆正常，第五关才触发传送，对应测试全部通过。

问题五：重开关卡后西楼出口仍处于锁定状态（庞绮君，E14）

发现过程：`WestBuildingLockTest` 的 `testRestartLevelResetsWestLock` 用例失败，复现步骤：第三关进入西楼触发困锁 → 超时失败重开 → 再次进入西楼，发现门仍处于锁定状态，但玩家背包已清空、没有锤子，导致无法脱困，形成死局。

原因分析：`LevelManager.restartCurrentLevel()` 调用时只重置了倒计时和背包，西楼出口锁定状态（`westBuildingExitLocked`）存在 `LevelManager` 的字段中，未被纳入重置逻辑。

解决方案：在 `restartCurrentLevel()` 中补充 `westBuildingExitLocked = false` 的重置，使关卡内各状态标志随重开一并清除。修复后重开关卡再进西楼，困锁状态可以正常重新触发，玩家有机会重新合成锤子脱困。

问题六：`sleep` 命令在第二关未检查 `submit` 完成状态就允许通关（彭慧星，E6）

发现过程：集成测试时发现，第二关玩家拿到一卡通后，不经过 `submit` 提交归寝单，直接进寝室执行 `sleep` 也能触发过关。这一问题在 `SleepCommandTest` 中由于测试构造数据时直接 `mock` 了关卡配置，单元测试没有捕获。

原因分析： SleepCommand 的通关检查顺序是：持一卡通 → 密码解锁（第五关）→ 触发过关，遗漏了对 isDormitorySubmitCompleted() 的判断，而 submit 完成状态是 LevelManager 单独维护的字段，两个命令没有显式联动。

解决方案： 在 SleepCommand 的通关前置条件中追加判断：如果当前关卡配置 requiresDormitorySubmit() 为 true 且 isDormitorySubmitCompleted() 为 false，则提示“请先提交归寝单”并返回。集成测试补充了“未 submit 直接 sleep 应被拒绝”的场景用例。

问题七：多人分支 Checkstyle import 顺序不一致导致 CI 失败

发现过程： feature/xmq 的 PR 合并进 dev 后，CI 的 mvn checkstyle:check 环节报错，错误集中在 import 语句顺序不符合 Checkstyle 规则——java.* 包放在 cn.* 之后、静态 import 没有单独分组。Git 历史中有两次专门修复 Checkstyle 的提交（fix: 修复 Checkstyle import 顺序以通过 CI）可以印证这一点。

原因分析： 各成员使用的 IDE 自动格式化规则不统一，IntelliJ 默认的 import 排序与项目配置的 Checkstyle 规则有出入，提交前没有养成在本地先跑 mvn checkstyle:check 的习惯。

解决方案： 统一在各成员 IDE 中导入项目的 Checkstyle 配置文件，并约定提交前本地执行一次 mvn checkstyle:check 确认通过。后期为 CI 在 Checkstyle 步骤失败时直接打印违规文件路径，方便快速定位。

问题八：联机模式下关卡重开/进入下一关时非主玩家背包未清空

发现过程： T6 联机回归测试阶段，两台设备联机通关第一关进入第二关后，发现房主背包已清空，但另一名玩家背包里第一关收集的物品仍然保留，影响后续关卡平衡。

原因分析： LevelManager.startLevel() 中清空背包的逻辑调用的是 game.getPlayer().dropAllItems()，而 game.getPlayer() 在联机模式下返回的是当前“活跃玩家”，非活跃的其他联机玩家没有被遍历。这个 Bug 在单机测试阶段无法复现，只有多人联机才会触发。

解决方案： 在 Game 类中增加 clearAllInventories() 方法，遍历 onlinePlayers 和 basePlayer 全部清空。LevelManager.startLevel() 改为调用

`game.clearAllInventories()` , 并在 `MultiplayerGameTest` 中补充了 `testCompleteLevelClearsAllPlayersInventory` 和 `testRestartSameLevelClearsAllPlayersInventory` 两个回归测试用例, 修复后均通过。

问题九: `feature/1j` 与 `feature/xmq` 合并时 `Game.java` 产生冲突

发现过程: 联机背包清空的修复 (`feature/1j`) 和服务端联机功能扩展 (`feature/xmq`) 都对 `Game.java` 做了改动, 两个分支同期并行开发, PR 合并时出现合并冲突, 冲突集中在 `addOnlinePlayer` 方法附近以及新增的 `clearAllInventories` 方法插入位置。

原因分析: 两个分支分别在 `Game.java` 的不同区域新增了方法, Git 无法自动判断合并顺序, 将冲突区域标记为手动解决。

解决方案: 两人在 PR 提交前通过对话确认各自改动的范围, 手动合并时保留双方改动, `clearAllInventories()` 插入在 `addOnlinePlayer` 之后。此后约定凡是需要修改 `Game.java` 这类核心类, 提前在群里通知其他成员, 避免大范围并行改动同一文件。

问题十: H2 存档测试在首次运行时因表不存在报错

发现过程: `GamePersistenceServiceTest` 在 CI 环境首次拉取代码后运行时抛出 `Table "GAME_SAVES" not found` 异常, 本地开发时因 H2 文件库已存在不会复现, 但 CI 每次都是干净环境。

原因分析: H2 数据库的建表 SQL 放在应用启动时执行, 但测试类直接构造 `GamePersistenceService` 实例, 绕过了 Spring 容器启动流程, 建表语句没有被执行, 测试就开始操作不存在的表。

解决方案: 在测试类的 `@Before` 方法中显式调用 `DatabaseProvider.initSchema()`, 保证每次测试前表结构都已就绪, 并改用 `jdbc:h2:mem:test` 内存库, 测试结束后自动丢弃, 不影响仓库中的任何数据文件。修复后 CI 连续通过。

问题十一: Vite 开发服务器仅绑定 `localhost`, 局域网联机演示失败

发现过程： 准备联机演示时，第二台机器在浏览器输入主机 IP（如 `http://10.76.24.214:5173`）无法访问 Vue 前端，但主机本机访问 `localhost:5173` 完全正常，Spring Boot 后端 8080 端口在局域网内可以正常访问。

原因分析： Vite 默认只监听 `127.0.0.1`（或 `:::1`）本地回环地址，局域网的其他设备发来的请求直接被操作系统丢弃，与防火墙无关。

解决方案： 启动命令改为 `npm run dev -- --host`，Vite 会绑定到本机所有网卡地址（`0.0.0.0`），控制台输出中的 Network 地址即为局域网可访问地址。演示前写入了 `scripts/run-multiplayer-server.ps1` 启动脚本，将 `--host` 参数固化，避免现场演示时遗漏。

小结：

以上问题可以归为三类：①单模块逻辑边界未考虑周全（问题一、二、五、六），这类问题通过单元测试覆盖边界分支基本可以提前发现；②多人协作引入的集成问题（问题七、八、九），这类问题需要在测试阶段多做跨模块的联合验证；③环境配置差异问题（问题十、十一），本地可运行但 CI 或演示环境暴露，需要在部署文档和启动脚本中把环境前提显式化。总体来看，越早在开发过程中建立本地 CI 门禁意识（本地跑 `mvn checkstyle:check + mvn test`），能明显减少联调阶段出现的低级问题。

6. 任务总结

本小组围绕文字冒险游戏《熄灯前归寝》，完成了五关单机主流程、Vue Web 图形界面、H2 存档与读档，以及至少两人参与的联机模式。共完成 GitHub Issue 27 项（功能 20 项、测试 7 项），后端单元测试类 30 余个，CI 门禁在 dev 分支持续保持绿灯状态。游戏五关主线可玩通，核心拓展命令均有单元测试覆盖，联机模式可演示。不足之处在于联机功能深度有限，Vue 前端未接入自动化测试，部分系统测试用例覆盖不够全面。

在技术层面，这次项目加深了对命令模式、分层架构、Spring Boot REST API 以及 Vue 3 Composition API 的理解，尤其是在多人协作场景下，架构分层清晰的价值体现得更为直接——各人负责的模块边界明确，并行开发时干扰较小。在软件过程层面，通过 GitHub Issue 系统化拆分任务并标注依赖关系，使整体进度可追踪；分支管理与 PR 评审机制让代码改动有记录、有人把关，Checkstyle 与 CI 门禁将格式规范和测试覆盖强制落地，减少了依赖口头约定的不确定性。评审过程中发现的若干问题，如职责划分不合理、关卡状态未重置等，也是纯靠单人测试难以暴露的。总体而言，这次实践让我们对“工程化协作”有了更具体的认识，理解了规范与工具在团队开发中的实际作用。

7. 参考文献

- [1] 涂风涛. 基于 Java 多线程技术的聊天系统设计与开发[J]. 科技广场, 2012, (9): 83-86.
- [2] 崔娟, 张生月, 章恒. Maven 与 Gradle 构建工具对比分析[J]. 科技创新与应用, 2025, 15(21): 93-96. DOI:10.19981/j.cn23-1581/G3. 2025. 21. 021.
- [3] 董威. 单元测试及测试工具的研究与应用[J]. 微型电脑应用, 2008, (5): 24-26+23+5.
- [4] 王科翔, 刘星含, 姜志城, 等. 基于 Spring Boot+Vue 框架的智慧学习平台的设计与实现[J]. 无线互联科技, 2023, 20(14): 65-68.
- [5] 周芯宇, 陈伟, 吴国全, 等. REST API 设计分析及实证研究[J]. 软件学报, 2022, 33(9): 3271-3296. DOI:10.13328/j.cnki.jos.006383.
- [6] 方生. 基于“Vue.js”前端框架技术的研究[J]. 电脑知识与技术, 2021, 17(19): 59-60+64. DOI:10.14004/j.cnki.ckt.2021.1863.
- [7] 石怡. 基于 MySQL 数据库的查询性能优化研究[J]. 四川职业技术学院学报, 2021, 31(1): 164-168. DOI:10.13974/j.cnki.51-1645/z. 2021. 01. 030.
- [8] 肖卓宇, 何镭, 陈果, 等. 带特征指标约束描述的设计模式分类挖掘[J]. 山东大学学报(工学版), 2020, 50(6): 48-58.
- [9] 李丹, 张师毅. 基于 JAVA 的仿 QQ 聊天系统的设计[J]. 微型机与应用, 2013, 32(24): 11-13. DOI:10.19358/j.issn.1674-7720. 2013. 24. 006.
- [10] 陈博, 周亦敏. 基于 Kubernetes 的 CI/CD 平台[J]. 计算机系统应用, 2020, 29(12): 268-271. DOI:10.15888/j.cnki.csa.007682.

《软件工程实践（二）》成绩评定表

专业、班级	软件 2301	小组名称	519club
代码仓库地址	https://github.com/wutkst/kai-fa-519club		
成员一姓名	肖梦琪	学号	1023004400
成员二姓名	刘晶	学号	1023004409
成员三姓名	彭慧星	学号	1023004402
成员四姓名	庞绮君	学号	4002510050
评价内容		满分	实得分
			得分
实践任务 完成情况	软件架构设计	25	
	软件版本计划与项目管理	25	
	软件分支管理	25	
	代码规范与代码评审	15	
	持续集成与部署	10	
实践任务评分		100	
实践报告 总评情况	学习态度与考勤	10	
	报告格式的规范性	20	
	报告的逻辑结构与语言表达	25	
	实践内容的正确性与合理性	40	
	文献引用及标注	5	
实践报告评分		100	
最终评定成绩（以优、良、中、及格、不及格评定）			

指导教师签字：

年 月 日